

Bachelor Thesis

On Markov Sequence Compression and Summarization in Transformers and Large Language Models



Fachbereich Informatik

Goethe-Universität Frankfurt am Main

Name: MORITZ CONSTANTIN WOSCH

Matriculation Number: 7760732

First examiner: Prof. Dr. Visvanathan Ramesh

Date of Submission: Wednesday 16th April, 2025

Bitte dieses Formular zusammen mit der Abschlussarbeit abgeben!

Erklärung zur Abschlussarbeit

gemäß § 35, Abs. 16 der Ordnung für den Bachelorstudiengang Informatik vom 17. Juni 2019:

Hiermit erkläre ich

(Nachname, Vorname)

Die vorliegende Arbeit habe ich selbstständig und ohne Benutzung anderer als der angegebenen Quellen und Hilfsmittel verfasst.

Ich bestätige außerdem, dass die vorliegende Arbeit nicht, auch nicht auszugsweise, für eine andere Prüfung oder Studienleistung verwendet wurde.

Zudem versichere ich, dass alle eingereichten schriftlichen gebundenen Versionen meiner vorliegenden Bachelorarbeit mit der digital eingereichten elektronischen Version meiner Bachelorarbeit übereinstimmen.

Frankfurt am Main, den

Unterschrift der/des Studierenden

Abstract

This thesis investigates the capabilities of Transformer models as statistical compressors by examining their abilities to estimate and reproduce sequences drawn from k th-order binary Markov chains. Through systematic experimentation with these synthetic sequences, it is evaluated how effectively Transformers function as statistical compressors of their training data. The study employs small Transformer architectures (2-4 layers) trained on sequences generated from Markov chains of orders 1-6. The findings demonstrate that Transformers achieve near-optimal performance in preserving patterns from lower-order Markov chains (1-4) while showing meaningful but degraded performance for higher-order chains (5-6). Minimum sequence lengths required for accurate pattern preservation for different Markov orders are established and the relationship between sequence length, complexity of the underlying Markov chain, and pattern preservation capabilities is quantified. The results suggest that Transformers function as statistical compressors of their training corpus, with implications for understanding their summarization capabilities through In-Context-Learning in NLP tasks. This work contributes to the theoretical foundation for developing stochastic models for text summarization and proposes future directions for investigating hierarchical structures in language through inhomogeneous Markov chains.

Zusammenfassung

Diese Arbeit untersucht Transformer-Modelle als statistische Kompressoren, insbesondere ihre Fähigkeit, Sequenzen aus binären Markov-Ketten der Ordnung k zu schätzen und zu reproduzieren. Durch systematische Experimente mit synthetischen Sequenzen wird evaluiert, wie effektiv Transformer die zugrundeliegenden statistischen Eigenschaften ihrer Trainingsdaten erfassen und repräsentieren können. Die Studie verwendet kleine Transformer-Architekturen (2-4 Schichten), die auf Sequenzen aus Markov-Ketten der Ordnungen 1-6 trainiert wurden. Die Ergebnisse zeigen, dass Transformer bei der Erlernung von Mustern aus Markov-Ketten niedrigerer Ordnung (1-4) nahezu optimale Leistungen erzielen, während sie bei höherer Ordnung (5-6) sinnvolle, aber weniger präzise Ergebnisse liefern. Für verschiedene Markov-Ordnungen wurden kritische Schwellenwerte der Sequenzlängen festgelegt und der Zusammenhang zwischen Sequenzlänge, Komplexität der zugrundeliegenden Markov-Kette und der Fähigkeit zur Mustererhaltung quantifiziert. Die Ergebnisse belegen, dass Transformer als statistische Kompressoren ihres Trainingskorpus fungieren, was weitreichende Implikationen für ihre Fähigkeit zur Textzusammenfassung durch In-Context-Learning hat. Diese Arbeit leistet einen Beitrag zur theoretischen Grundlage stochastischer Modelle der Textzusammenfassung und eröffnet neue Forschungsperspektiven zur Untersuchung hierarchischer Sprachstrukturen mittels inhomogener Markov-Ketten.

Acknowledgments

I want to express my sincere gratitude to Professor Ramesh for his guidance and mentorship throughout this thesis. His enthusiasm for discussions about the bigger questions around this work and his constant encouragement to maintain attention to detail have significantly shaped this thesis and my approach to research. I am particularly grateful to Gamze Akyol and Achref Jaziri for their research expertise and support with technical issues. The deepest thanks go to my family and friends, whose support and encouragement have been constant sources of motivation for me.

This work would not have been possible without the contribution of all these people.

Contents

1	Introduction	12
1.1	Background	12
1.2	Motivation and Research Objective	12
1.3	Thesis Structure	13
2	Preliminaries	15
2.1	From Natural Data to Statistical Models	15
2.2	Markov Chains	15
2.3	Overview of NLP	17
2.3.1	Rule-Based Systems	18
2.3.2	Statistical Models	18
2.3.3	Neural Approaches	19
2.4	Introduction to Transformers	20
3	Literature Review	24
3.1	Characterization of a Summary	24
3.2	Interpretability of LLMs	25
3.3	Transformers as Compressors	26
4	Approach	28
5	Experiment Setup	30
5.1	Intuition and Goal	30
5.2	Metrics	30
5.2.1	Kullback-Leibler Divergence	30
5.2.2	Mean Absolute Error	31
5.3	Markov Functions	31
5.4	Transformer Model	32
5.5	Experiment Pipeline	32
6	Results and Discussion	34
6.1	Training Dynamics	34
6.2	Output Performance and Error Distribution	35
6.3	Comparative Evaluation	36
6.3.1	Random vs. Transformer Performance	36
6.3.2	Lower Bound vs. Transformer Performance	37
6.4	Statistical Validation	38
6.4.1	Performance Stability	38
6.5	Error Analysis	38
6.5.1	Performance for Order 2	38
6.5.2	Performance for Orders 5 and 6	39
7	Conclusion	41
7.1	Contributions	41
7.2	Future Work	41
A	Tables	49

Contents	10
B Figures	52
C Code Listings	55

Acronyms

AI Artificial Intelligence. 12, 18, 24, 25

ANN Artificial Neural Network. 19, 25, 26

BPE Byte-Pair Encoding. 20

FNN Feed-Forward Neural Network. 22, 23

HMM Hidden Markov Model. 18, 19

ICL In-Context Learning. 25, 27, 30

KLD Kullback-Leibler Divergence. 30–32, 35–41, 43, 53, 54

LLM Large Language Model. 12–14, 24–28, 41, 42

MAE Mean Absolute Error. 31, 35–39, 41, 43, 53, 54

NLP Natural Language Processing. 12, 13, 15, 17–20

RNN Recurrent Neural Network. 19

1 Introduction

1.1 Background

Large Language Models (LLMs), based on the Transformer Architecture, have experienced rapid growth in recent years and have become increasingly integrated into everyday applications. Natural Language Processing (NLP) has become one of the most popular and research-heavy fields of Artificial Intelligence (AI). Initially developed for machine translation-oriented tasks, LLMs quickly evolved to mimicking human language through next-word prediction and have since demonstrated remarkable improvements in various domains, driven by innovations in their architecture and scaling of parameter sizes. The Transformer architecture, introduced by Vaswani et al. (2017), replaced the sequential processing of inputs in earlier models and introduced a self-attention mechanism, enabling parallel processing of word pieces to incorporate contextual relations of words. This architecture allows LLMs to capture long-range dependencies and relationships in text more effectively. Furthermore, scaling laws revealed that increasing the number of parameters and training data leads to predictable performance improvements (Kaplan et al. 2020), making scalability a key aspect of LLMs.

The capabilities of these models range from basic natural language understanding (Devlin et al. 2019) to meaningful text generation and continuation (Radford et al. 2019), sentiment analysis (Sun, L. Huang, and Qiu 2019), text summarization (Pu, Gao, and Wan 2023), and a host of other advanced language processing capabilities. Despite the fact that the underlying principles and architectures of these models are well-known, their behavior is often complex and counterintuitive to grasp. LLMs are frequently described as black boxes, meaning that they produce reasonable and meaningful outputs while the exact mechanisms of the billions of parameters that work within the model are neglected as too complex to interpret for humans.

Unlike deterministic systems designed by humans, where variables have explicitly encoded purposes, large Transformers encode patterns and relationships within billions of parameters without trivial interpretability. This thesis investigates how Transformers can be systematically evaluated as statistical compressors by training them on synthetic data generated by Markov chains. The use of Markov chains provides a controlled environment where ground-truth statistical properties are known, allowing insights into the mechanisms underlying pattern preservation in Transformers.

1.2 Motivation and Research Objective

The idea behind this thesis lies in the exploration of the following question: *How can we systematically evaluate the mechanisms of a Transformer model?*

To understand what occurs within a complex system, it is essential to identify reasonable and well-defined concepts that can potentially explain its behavior. By clearly articulating these ideas and testing the models against them through quantitative experiments, this work aims to establish and strengthen the understanding of Transformers and support specific hypotheses about their inner workings. Fundamental insights on this architecture can lead to an understanding of the heavily up-scaled Transformers that are LLMs. The motivation for being able to explain AI-systems lies in demystifying their behaviors, limitations, and social impacts, therefore reducing risks for downstream applications and allowing the improvement of these systems (Zhao et al. 2023) .

This thesis takes a focused perspective on how Transformers are able to estimate simple stochastic models. More specifically, this work focuses on the research objective: *How accurately can a Transformer compress and reproduce the statistical patterns present in its training corpus?* By training models on sequences generated from Markov chains with known properties, one can precisely measure how well the

models capture and generate text according to these ground-truth probability distributions. Although Transformers are algorithmic models that estimate the relations of words to each other from large datasets, they show substantial abilities in preserving patterns and rules. For example, these models demonstrate the ability to generate and analyze computer code (Jiang et al. 2024), as well as conserving structural constraints such as Markov chains (Zekri et al. 2024), N -grams (Nguyen 2024), or syntactic hierarchies (Veres 2022). This suggests that LLMs can maintain and manipulate symbolic and stochastic systems that are implicitly embedded in language.

The larger context of these implications is text summarization. While it has been shown that LLMs exceed in summarizing text (Pu, Gao, and Wan 2023), it is not fully understood which mechanisms lie behind this process. A summary is a concise representation of a text that captures the main ideas and concepts in the target text while compressing the data. Given the premise that LLMs seem to encapsulate structural constraints from their input, it suggests that they internalize context in a way that aligns with stochastic systems. The hypothesis is: **Transformers can be viewed as high-level statistical compressors of their training corpus.** If LLMs consistently preserve input structures, this will enhance understanding of how they handle context, which is crucial for optimizing and trusting model output. Furthermore, this might give insight into the very nature of language itself. The comparison of statistical compression with text summarization might raise questions about the computation of information in language.

By examining compression tasks with Markov-generated sequences, this work aims to explore the extent to which LLMs can estimate and manipulate simple stochastic systems, contributing to the interpretability of Transformer-based systems. It also seeks to determine whether these models can maintain consistency with the original generative framework, improving the understanding of their capabilities in stochastic modeling and structured language processing. This capability might relate statistical systems to neural ones in NLP.

1.3 Thesis Structure

This thesis is organized into seven sections. Following this introduction, Section 2 lays the foundations for the study by providing an overview of statistical data modeling, Markov chains, NLP and the mechanisms of the Transformer architecture. Section 3 presents a detailed review of the relevant literature, focusing on recent scientific studies related to automated summaries, the interpretability of complex systems, and establishes the hypothesis that Transformers can be viewed as compressors of their training corpus. Section 4 outlines the scientific methodology in this research, while Section 5 describes the experimental setup, including the models and the dataset used in this approach. The results of the experiments are presented, analyzed, and discussed in Section 6, followed by an in-depth error analysis. Finally, Section 7 concludes the thesis by summarizing the key findings, discussing their implications, and proposing directions for future research.

Notation

Throughout this thesis, the following mathematical notation is used:

- Vectors are denoted in bold lowercase (e.g., $\mathbf{x}, \mathbf{q}$), matrices in bold uppercase (e.g., $\mathbf{M}, \hat{\mathbf{M}}$).
- Random variable sequences are denoted by calligraphic letters (e.g. $\mathcal{X} = (X_1, \dots, X_n)$), while observed sequences (realizations) are denoted by lowercase tuples (e.g. $(x_1, \dots, x_n)$). The state space from which symbols are drawn is denoted by $\mathcal{S}$.
- For probability distributions and statistical measures, the notation is $P(\cdot)$ for probabilities and $E[\cdot]$ for expectations.

Note on terminology: Throughout this thesis, the term *Transformer* refers to the neural network architecture introduced by Vaswani et al. (2017). The term *Large Language Model* refers to large-scale Transformer-based models that are trained on large text corpora, such as GPT-4 or Llama. Although all LLMs are Transformers, not all Transformers are LLMs.

2 Preliminaries

This section aims to give an overview on sequences as natural data and statistical models such as Markov chains that can approximate data. It further introduces the evolution of approaches in the area of NLP and the mechanics of modern Transformer-based models which provides necessary knowledge for the systematic evaluation conducted in this thesis.

2.1 From Natural Data to Statistical Models

Natural data is produced from complex processes whose underlying mechanisms are often unknown or only partially understood. As Breiman (2001) notes in his work on statistical modeling cultures, natural data can be viewed as being generated by a black box where predictor variables (input) enter and response variables (output) are produced, with nature's internal workings hidden from direct observation.

A data structure considered in this work is a sequence of symbols $\mathcal{X} = (X_1, \dots, X_n)$, drawn from some state space $\mathcal{S}$. In the context of language, this might be a population of words drawn from the English alphabet that form a text. The generating process behind the data remains unknown, which prevents one from making meaningful predictions on the output of this black box. However, one can attempt to capture and understand this mechanism through the lens of a statistical model. This model can be described as a set of assumptions about a problem domain that is expressed in a precise mathematical form. Hence, the potentially complex and high-dimensional natural data gets *compressed* into a stochastic model characterized by a relatively small number of parameters.

Let $\mathbf{P}$ be the unknown natural probability distribution that generates the data, Θ be the parameter space of the stochastic model, and let $\mathcal{M} = \{\mathbf{P}_\theta : \theta \in \Theta\}$ be the family of probability distributions defined by this model. The goal is to find parameters θ^* such that $\mathbf{P}_{\theta^*}$ approximates the true underlying distribution $\mathbf{P}$ in an optimal sense. This compression of complex natural data into a parametric stochastic model enables one to make *predictions* about future predictions in the same process, while it also provides *insight* into the underlying mechanism generating the data. Once a suitable model is fitted, it can be used to synthesize new sequences that exhibit properties similar to the natural data. This represents a transformation from the black box of nature to an interpretable statistical framework. The quality of this approach depends on the model's ability to capture the relevant patterns while remaining interpretable. The challenge lies in finding the right interplay of model complexity and interpretability while ensuring that the model fits the data representatively. Breiman (2001) labels this paradigm of modeling nature by stochastic systems as the "data modeling culture".

2.2 Markov Chains

The statistical models considered in this work belong to the family of Markov processes (Seabrook and Wiskott 2023), an elementary group of stochastic models that describe the temporal evolution of an infinite sequence of random variables $\mathcal{X} = \{X_t : t \in T\}$, defined on a state space $\mathcal{S}$ and indexed by a time set T . A k th-order Markov chain is a Markov process on discrete time (i.e., $T = \mathbb{N}_0$) describing a sequence of possible states on $\mathcal{S}$ in which the probability of each state depends only on the previous k states. This dependency is described as the *Markov property*. For instance, in a 1st-order Markov chain, the transition probabilities from one state to another depend only on the current state, not on the sequence of past states. Let $\mathcal{X}$ be a sequence of random variables representing the states of the Markov chain. The process is a 1st-order Markov chain if:

$$P(X_t = x | X_{t-1} = x_{t-1}, X_{t-2} = x_{t-2}, \dots, X_1 = x_1) \quad (1)$$

$$= P(X_t = x | X_{t-1} = x_{t-1}) \quad (2)$$

for all $t \geq 1$ and all states $x_1, \dots, x_t$ in $\mathcal{S}$.

A Markov chain with memory (where $k > 1$) satisfies:

$$P(X_t = x | X_{t-1} = x_{t-1}, X_{t-2} = x_{t-2}, \dots, X_1 = x_1) \quad (3)$$

$$= P(X_t = x | X_{t-1} = x_{t-1}, X_{t-2} = x_{t-2}, \dots, X_{t-k} = x_{t-k}) \quad (4)$$

for $t > k$. In other words, the future state depends on the past k states.

A Markov chain is *homogeneous* if the transition probabilities between states are fixed and do not change over time. Hence, the system dynamics are consistent. For *inhomogeneous* Markov chains, the transition probabilities can vary over time. This allows the system dynamics to change with time steps. Let $N = |\mathcal{S}|$ be the size of the state space. A homogeneous k th-order Markov chain can be fully described by a $N^k \times N$ -sized matrix where each row corresponds to a unique combination of the previous k states and each column corresponds to a possible next state in $\mathcal{S}$. For instance, a 1st-order Markov chain can be represented by an $N \times N$ -sized stochastic matrix:

$$\mathbf{M} = \begin{pmatrix} P_{11} & P_{12} & \cdots & P_{1N} \\ P_{21} & P_{22} & \cdots & P_{2N} \\ \vdots & \vdots & \ddots & \vdots \\ P_{N1} & P_{N2} & \cdots & P_{NN} \end{pmatrix} \quad (5)$$

which is called the *transition matrix* of the Markov chain. Each row corresponds to one possible combination of the previous state, and each column represents a possible next state. The matrix has the property: $\sum_{j=1}^N P_{i,j} = 1$ for all $i \in \{1, \dots, N^k\}$, which means that each row represents a probability distribution over the possible next states.

The stationary distribution of a Markov chain is a probability distribution that remains unchanged over time. This can be interpreted as the long-term proportion of time the Markov chain spends in each state. It is represented as a row vector π whose entries are probabilities that sum to 1, and given the transition matrix $\mathbf{M}$, it satisfies $\pi = \pi \mathbf{M}$. All finite state Markov chains that are irreducible and aperiodic¹ have a definite stationary distribution.

This work only deals with homogeneous, discrete-time Markov chains of countable state spaces. In the context of sequence generation, such chains can be used to create synthetic sequences over the given transition probabilities. A simple example of such a Markov chain of order 1 that is able to generate a sentence can be seen in Figure 1.

This chain will output either "This is my grandmother." or add a probabilistic amount of repetitions of "great-" before "grandmother". The transition probabilities from "great-" to "grandmother" (70%) do not change with the number of repetitions of the state "great-". This emphasizes the memorylessness of the chain. Transitions that are not explicitly shown in the figure have probability 0.

The previously defined Markov chains can be practically estimated from observed sequences using conditional N -gram models² with $N = k + 1$ (Rajaraman et al. 2024). Given an observed sequence $(x_1, \dots, x_n)$ in $\mathcal{S}^n$, the conditional k -gram estimator $\hat{\mathbb{P}}_k(\cdot | x_1, \dots, x_n)$ estimates the probability of observing a symbol conditioned on the previous k symbols. For $x \in \mathcal{S}$:

$$\hat{P}_k(x | x_1, \dots, x_n) \triangleq \frac{\sum_{i=k+1}^n \mathbb{I}(x_i = x, x_{i-1} = x_n, \dots, x_{i-k} = x_{n-k+1})}{\sum_{i=k+1}^n \mathbb{I}(x_{i-1} = x_n, \dots, x_{i-k} = x_{n-k+1})}, \quad (6)$$

¹A Markov chain is irreducible if it is possible to reach any state from any other state in a finite number of steps. It is aperiodic if it does not exhibit periodic cycles in returning to states.

²An N -gram is a sequence of N adjacent symbols in particular order drawn from a given sequence. An N -gram model is essentially equivalent to an $(N - 1)$ -order Markov chain, as it estimates the probability of the next symbols in a sequence based on the preceding N -gram.

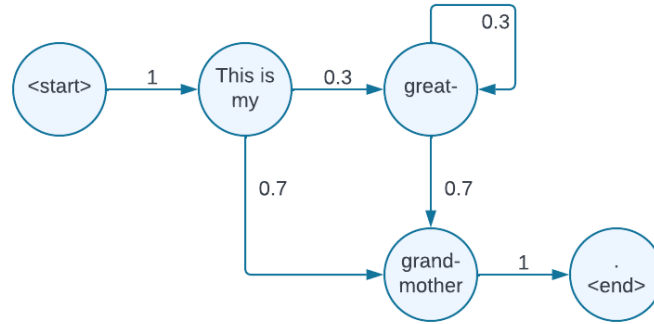


Figure 1: An example for a 1st-order Markov chain, where the generated text is represented by the labels on the nodes.

where $\mathbb{I}(\cdot)$ is the indicator function. This estimator corresponds to the maximum-likelihood estimate of the transition probabilities of a k th-order Markov chain and is defined whenever the denominator is non-zero. Applying this estimation to a text, one can estimate Markov properties for this text. For instance, a 2nd-order Markov transition matrix can be estimated for the full drama 'Hamlet' by Shakespeare. Here, the symbols depict single words.

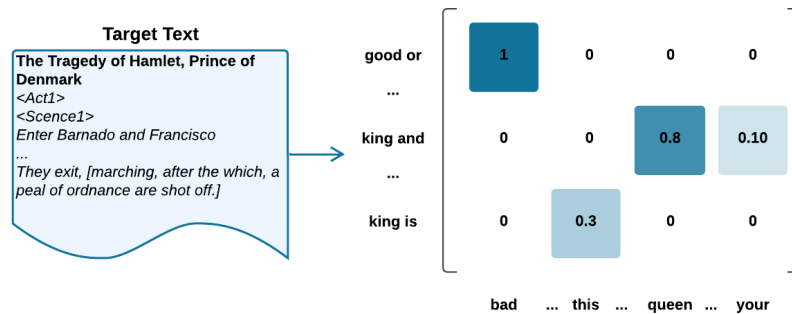


Figure 2: An illustration of a 2nd-order Markov chain that is estimated for Shakespeare's Hamlet. The rows show all possible sequences of size $k = 2$ while the columns represent all existing words in the document. The cells show how likely the word in the column follows the context in the row.

The transition probabilities for a given state are usually more uniform for lower k and become more concentrated as k grows, since longer sequences provide more specific context, reducing ambiguity in the transitions. For instance, in Shakespeare's Hamlet, the 2-gram "king and" will likely be followed by "queen" - in both the original document and a summary, as this is a strong contextual association. However, the 1-gram "and" will presumably exhibit a broader range of possible succeeding words, as it is less contextually constrained. This illustrates how higher-order Markov chains capture more precise relationships.

2.3 Overview of NLP

NLP deals with a highly complex form of natural data produced by human beings: natural language. Language can be viewed as sequences of symbols (words, characters, etc.), produced by human cognition. The challenge of NLP lies in developing models that can capture and reproduce the patterns and structures (syntax) inherent in these sequences while dealing with the non-trivial concept of meaning (semantics). The following subsection describes the scientific evolution of attempts to model language as natural data.

2.3.1 Rule-Based Systems

Viewing human language as a "system" goes back to the early 20th century, when Swiss linguist Ferdinand de Saussure introduced the idea that language is "a system of signs that express ideas" (Saussure 1916). He claimed that a sign consists of a signifier (a sound or image) and a signified (the concept which the signifier represents), while the relationship between the two is arbitrary. This notion contributed significantly to understanding of language because it laid the groundwork for theories of language structure and computational approaches to linguistic analysis. Chomsky (1957) extended these ideas in the 1950s by introducing the *Generative Phrase Structure Grammar*. The general idea was developing a method to divide an expression into its immediate constituents until one obtains syntactically indivisible parts. These constituents can then be combined by transformations to form sentences.

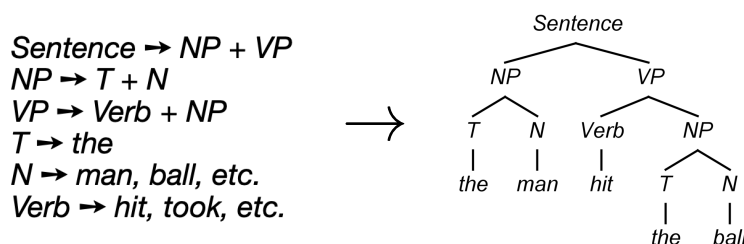


Figure 3: An example for a Phrase Structure Grammar and a sentence produced by it (Veres 2022).

The grammar was intended to be fully understood by a Turing Machine³ and consists of rewrite rules of the form *class label* $\rightarrow$ *class label/string*. It provides a finite set of sentences, each finite in length (as long as it is constructed of a finite set of elements). An example of such a grammar can be seen in Figure 3. The majority of NLP approaches in that time relied on such rule-based systems. The Georgetown-IBM experiment in 1954 demonstrated a Russian-English machine translation system using only hand-crafted rules (Hutchins 2004), Weizenbaum (1966) enabled humans to chat with 'ELIZA', a psychiatrist imitation program using a pattern matching and substitution methodology. Symbolic AI-systems demonstrated fundamental language generating capabilities but they quickly reached their limitations caused by numerous drawbacks: They require large and complex rule sets that are difficult and costly to modify and maintain, this also makes them very difficult to scale, they struggle to generalize (i.e., adapting and evolving rules designed for one language domain to another), and they are not robust to perturbed input⁴. In conclusion, purely rule-based systems are able to generate syntactically correct outputs and can be used for very narrow basic tasks, but they do not seem to be capable of handling the ambiguities of natural language.

2.3.2 Statistical Models

In the 1980s the general approach experienced a paradigm shift: Instead of focusing on rules and human intuition, newer concepts relied on the statistical patterns of language. Apart from language being believed to be best explained by certain rules, the availability of extensive text corpora and increasing computational power opened up new possibilities. As estimated by Claude Shannon, about 60% of English language is redundant, meaning that new words can be predicted by patterns in a context (Frick and Sumby 1952). A Hidden Markov Model (HMM) (Petrie 1966) is a foundational probabilistic model and based upon a Markov chain representing sequences of observable events (e.g., words in a

³A Turing Machine is a theoretical computational model introduced by Alan Turing in 1936 that consists of an infinite tape and a set of rules for reading, writing, and moving along the tape, used to formalize the concept of computational power and algorithms.

⁴Perturbed input refers to a modified version of an original input where small changes ("perturbations") are introduced, such as altered words, added noise, or rephrased text.

sentence) generated by underlying hidden (unobservable) states (e.g., parts of speech). These models were particularly effective for named entity recognition and speech recognition, where sequences and their underlying structure are essential. However, HMMs are limited by their design, since a hidden state can only influence its observable state and the next hidden state. This constraint disables them from capturing long-range dependencies or contextual relationships in Natural Language. In the 1990s N -gram models gained popularity due to their accuracy in Natural Language tasks in spite of their simplicity and low computational requirements.

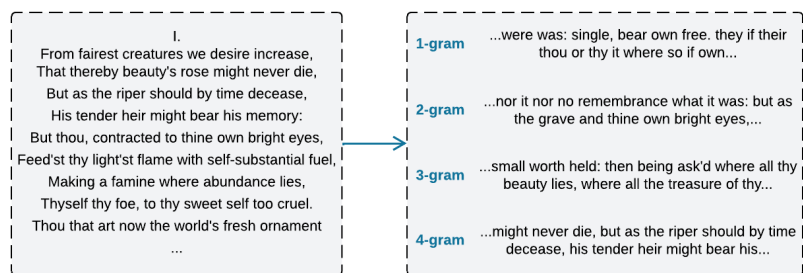


Figure 4: Sentences generated from four word-level N -grams computed from a collection of Shakespeare's sonnets. All characters were mapped to lower-case and symbols were treated as independent words. The sentences were created using Listing 1.

These models are based on Markov chains as they predict the next item in a sequence, typically words in a sentence, based on the previous $N - 1$ items, thus decomposing the probability of a sequence into a product of conditional probabilities, considering only a selected quantity of preceding items. An example for N -gram-generated text can be seen in Figure 4.

2.3.3 Neural Approaches

While earlier methods aimed to compress the data-generating process of language into handcrafted knowledge and statistical models for prediction and understanding, language poses highly complex challenges that highlight the complexity of natural phenomena. The non-trivial dependencies between syntax and semantics, combined with the contextual nature of meaning, make language generation a process that appears too complex to solve with simple data modeling. This limitation led to the development of approaches that attempt to directly estimate the input-output relationships of given language datasets while not making assumptions on the underlying data-generating process. Vast amounts of data are fed into the parameters of these algorithmic systems, thus, they can be viewed as estimators of the given data, their training corpus. Breiman (2001) labels such paradigms as the "algorithmic modeling culture".

In the 1940s McCulloch and Pitts (1943) had shown that it was possible to model the behavior of networks of neurons using a logical calculus. This inspired the usage of networks of artificial neurons with complex computational properties: Artificial Neural Networks (ANNs). These were developed into multi-layer learning networks which we understand as *Deep Neural Networks*. Deep Neural Networks were able to learn complex non-linear transformations across various domains: Computer Vision, NLP, pattern recognition, and more (Wason 2018). The notion was that ANNs are able to learn the complex cognitive phenomenon of language, without the need for symbolic abstractions. Recurrent Neural Networks (RNNs) are neural networks designed to process sequential data by maintaining a hidden state that conserve previous latent variables. Unlike traditional feed-forward networks, RNNs can process variable-length sequences, making them suitable for tasks such as speech recognition, time-series analysis, and NLP.

However, for larger inputs, these models tend to exhibit vanishing or exploding gradients⁵. Long Short-Term Memory networks were introduced in the late 90s to solve these problems by the use of gate units (Hochreiter and Schmidhuber 1997) and found their niche for voice and text processing. High complexity and computational costs remained. Bengio et al. (2003) proposed a neural language model, using a feed-forward neural network. The feed-forward neural network describes an artificial neural network that does not use connections to form cycles. Here, the data moves only in one direction, from the input nodes, through any hidden nodes, to the output nodes. The feed-forward neural network has no cycles or loops, reducing complexity. The goal was to model the joint probability function of sequences of words in a language by learning a distributed representation of words. Bahdanau, Cho, and Bengio (2015) introduced the attention mechanism that allows the model to focus on specific parts of the input sequence when generating each word in the output sequence. This contrasted with earlier models, where the encoder compressed the entire input sequence into a single fixed-length vector (the “context vector”), limiting their ability to handle long and complex sequences. In 2017, Vaswani et al. (2017) expanded on this, introducing the Transformer architecture, which greatly influenced the landscape of NLP by replacing recurrence and convolution with a Multi-Head attention mechanisms, enabling efficient parallel processing and the modeling of long-range dependencies in sequences. The mechanics of the Transformer will be elaborated in the next section.

2.4 Introduction to Transformers

A Transformer processes sequences in parallel by using solely self-attention mechanisms and non-linear feed-forward neural networks to model relationships between observed words. It enables the capture of long-range dependencies without relying on recurrence or convolution. A transformer is able to preserve contextual dependencies across entire sequences while reducing complexity and with relatively low computational resources compared to older models (Robine et al. 2023). The process from input embeddings to output probabilities is sketched in Figure 5. The Transformer is able to generate output probabilities for a next-predicted word-piece. This mechanism can be described as a mapping function of the model parameters (derived from its training corpus) and an input vector-space to an output vector-space. The essential steps in this system will be outlined in this subsection, as they are fundamental to comprehending the approach taken in this study.

Tokenization

The process of tokenization describes breaking down an input text into smaller units (tokens) that can be phrases, words, sub-words, or characters. This is a crucial step in the Transformer pipeline, as it affects how the model interprets and processes text data.

The two types of tokenization that are relevant for this thesis are:

1. Character Tokenizer: treats every single character in a sequence as a token.
2. Byte-Pair Encoding (BPE): merges the most frequent pairs of bytes in a dataset to create a vocabulary.

Embedding

In the embedding process the tokens generated by the Tokenizer get represented by a high-dimensional numerical vector. These embeddings capture the semantic meaning of words based upon the statistical relationship of tokens to each other in the training corpus. This way, tokens with similar meaning (that appear in similar context in the corpus) as "small" and "tiny" are spatially closer to each other in the high-dimensional vector space than tokens with opposite meanings as "small" and "huge". Initial

⁵Parameters that are fed into a neural net (gradients) become too small (vanishing) or too large (exploding) due to repeated multiplication in back-propagation.

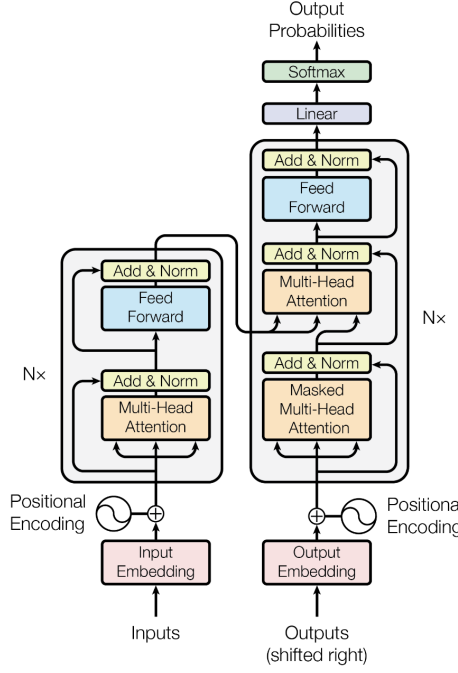


Figure 5: The original Transformer model architecture (Vaswani et al. 2017).

embeddings represent tokens without considering their context in a sequence. This way, words with multiple meanings (e.g. "current") only have one embedding.

Positional Encoding

The token embedding only provides information about the semantic meaning of a token. After the positional encoding the vectors include information about the order of tokens in the input sequence. Since Transformers process all tokens in parallel, they lack a built-in way to capture sequence order, unlike recurrent models. Without positional encoding the Transformer would treat the tokens in a sequence as an unordered set of pieces, while the order of words in a sentence is highly relevant (e.g. "A cat saw a mouse" has a different meaning than "A mouse saw a cat").

Attention Mechanism

The embedded tokens in a given sequence $\mathcal{E} = (\mathbf{e}_1, \dots, \mathbf{e}_T)$ now represent their general semantic meaning and their position in the input sequence. In language however, the context is essential in the meaning of a word. The self-attention mechanism (also called scaled dot-product attention) allows the model to dynamically focus on the most relevant parts of the input sequence when processing each token. Thus, *attention weights* are assigned to each token, helping the model capture relationships and dependencies, across short and long distances. Each embedded token $\mathbf{e}_i \in \mathbb{R}^{1 \times d_{\text{model}}}$ is transformed into three distinct vectors: a **query** vector $\mathbf{q}_i$, a **key** vector $\mathbf{k}_i$, and a **value** vector $\mathbf{v}_i$. These vectors are computed using learned projection matrices:

$$\mathbf{q}_i = \mathbf{e}_i \mathbf{W}_Q, \quad \mathbf{k}_i = \mathbf{e}_i \mathbf{W}_K, \quad \mathbf{v}_i = \mathbf{e}_i \mathbf{W}_V, \quad (7)$$

where $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V \in \mathbb{R}^{d_{\text{model}} \times d_k}$ are parameter matrices learned during training, and d_k is the dimensionality of the query and key vectors. These vectors allow the model to determine which tokens

"attend"⁶ to each other. The attention function is denoted as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V, \quad (8)$$

where $Q, K, V \in \mathbb{R}^{T \times d_k}$ are matrices containing the query, key, and value vectors for all tokens in the sequence, T is the sequence length (in tokens), and softmax is the softmax function⁷. To calculate how much each token attends to others, the attention mechanism computes the dot product of the query (Q) and key (K) matrices. These scores are scaled by the square root of the dimensionality of the key vectors, $\sqrt{d_k}$, to stabilize the gradients. The scaled scores are then normalized using the softmax function to obtain attention weights. Finally, the attention weights are multiplied by the value matrix (V), producing a new vector representation $\mathbf{e}'_i \in \mathbb{R}^{1 \times d_k}$ for each token. This new representation is enriched by the context provided by attending to other tokens in the sequence. For instance, the vector for "creature" in the sequence "a fluffy creature" is enriched by the adjective "fluffy" to embed a new vector that represents a "creature" that is "fluffy".

The process described above happens in one head of attention. In practice, multiple attention heads are used, each with its own learned projection matrices $\mathbf{W}_Q^h$, $\mathbf{W}_K^h$, and $\mathbf{W}_V^h$, to allow the model to capture different aspects of token relationships. Each head independently computes its own attention output, and the results are concatenated and projected back into the model dimension:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \text{head}_2, \dots, \text{head}_h)\mathbf{W}_O, \quad (9)$$

where each head is computed as:

$$\text{head}_i = \text{Attention}(Q\mathbf{W}_Q^i, K\mathbf{W}_K^i, V\mathbf{W}_V^i), \quad (10)$$

with learned projection matrices $\mathbf{W}_Q^i, \mathbf{W}_K^i, \mathbf{W}_V^i \in \mathbb{R}^{d_{\text{model}} \times d_k}$, and the output projection matrix $\mathbf{W}_O \in \mathbb{R}^{hd_k \times d_{\text{model}}}$, where $\mathbf{W}_O$ is the learned output projection matrix.

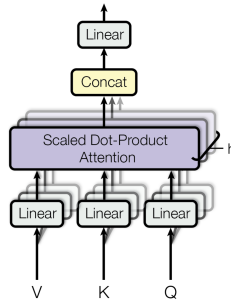


Figure 6: The Multi-Head Attention Mechanism (Vaswani et al. 2017).

Feed-Forward

After the Multi-Head Attention process each context-enriched token embedding is passed independently through a position-wise Feed-Forward Neural Network (FNN). This mechanism allows the network to learn and store patterns that are recognizable as factual recall. The multi-layer FNN consists of two linear layers separated by a non-linear activation function (ReLU⁸). Let $\mathbf{x} \in \mathbb{R}^{d_{\text{model}}}$ be a token embedding, the

⁶A token "attends" strongly to another token if it significantly influences its semantic meaning. For example, the token "large" strongly influences the meaning of the token "house" in the phrase "a large house".

⁷For an input vector $\mathbf{z} = [z_1, \dots, z_k] \in \mathbb{R}^k$, the softmax function outputs a probability vector $\mathbf{p} = [p_1, \dots, p_k] \in \mathbb{R}^k$, where higher input values correspond to relatively higher probabilities, and all probabilities sum to 1.

⁸The rectified linear unit (ReLU) is a function that outputs the input value if its positive and else 0.

feed-forward operation is defined as:

$$\text{FFN}(\mathbf{x}) = \text{ReLU}(\mathbf{x}\mathbf{W}_1 + \mathbf{b}_1)\mathbf{W}_2 + \mathbf{b}_2, \quad (11)$$

where $\mathbf{W}_1 \in \mathbb{R}^{d_{\text{model}} \times d_{ff}}$, $\mathbf{W}_2 \in \mathbb{R}^{d_{ff} \times d_{\text{model}}}$ are learned weight matrices, and $\mathbf{b}_1 \in \mathbb{R}^{d_{ff}}$, $\mathbf{b}_2 \in \mathbb{R}^{d_{\text{model}}}$ are learned bias vectors. Typically, the inner-layer dimension d_{ff} is larger than the model dimension d_{model} . The precise functionality of the FNN is non-trivial. According to Geva et al. (2021) FNNs function as key-value memories, where keys capture specific patterns from the input, and values generate predictions about the next token.

Decoding

At the output of the transformer stack each token is enriched with information from the entire sequence, effectively capturing the relationships and dependencies from the context. The embedding of the last token in the sequence ($\mathbf{x} \in \mathbb{R}^{d_{\text{model}}}$) is essential for the next-token prediction as it encodes the meaning of the entire sequence up to that point. This embedding is mapped to a vector of *logits* (raw scores) with dimensions equal to the vocabulary size. The logits vector is then transformed into a probability distribution over the vocabulary using the softmax function with a temperature parameter τ . The temperature parameter affects the randomness of the generated output. Lower temperatures make the model more deterministic by concentrating probability mass on the higher scoring tokens, while higher temperatures increase randomness by distributing probabilities more evenly across all tokens.

Finally, a newly generated token is sampled randomly from this probability distribution and appended to the input sequence. This process repeats iteratively until a termination token is generated.

Transformer models present an algorithmic approach to estimate the relationship of input values to output values in a training corpus. The nature and quality of the corpus is crucial here, as the Transformer always works as a function of its corpus, that is embedded into its parameters (Veres 2022). In consideration of the data modeling culture where stochastic models compress a natural data-generating process, this thesis argues that Transformers work as complex compressors of their corpus.

3 Literature Review

It has been clarified what statistical data modeling in terms of Markov chain is and how algorithmic approaches with regards to the Transformer architecture function. The comparison between the two will further be expanded on the theories of text summarization. This section discusses different approaches and empirical evidences about text summaries, the area of interpretability of AI-systems, and the concepts of how a LLM preserves patterns and extracts concepts. It provides necessary thoughts and ideas to argue this thesis.

3.1 Characterization of a Summary

While statistical compression is a method of reducing the amount of space needed to store or transmit data by analyzing its statistical properties, natural language summarization relates to this concept but in a way that goes beyond solely statistical properties. Summarization describes the process of compressing information from a larger dataset or document into a shorter, more compact one while retaining the essential ideas. It is crucial in managing the large amounts of data available today. Exploring the principles of human text summarization and automated approaches before the introduction of LLMs will be helpful in understanding the transition from statistical compression to text summarization.

There are various characterizations of what makes a summary a "high-quality" one. Some of which are **conciseness** (well-developed, concise paragraphs that are logically structured), information overlap with the target text, factual consistency, and language fluency and quality. Summarization demonstrates significant benefits for learning and long-term information retention (Allahyari et al. 2017), as it aligns with the way humans process and organize information. However, automating this task presents a challenge. Summarization requires managing the complex task of identifying relevant content, condensing it accurately, while maintaining coherence - skills that are inherently nuanced and difficult to replicate algorithmically. According to Allahyari et al. (2017) non-neural methods can be divided into **extractive** and **abstractive** paradigms. Extractive summarization means selecting key sentences or words directly from the source text and implementing them in a summary. This infers a somewhat *syntactic* summarization. Abstractive summarization, on the other hand, is about generating new sequences that only capture the essence of the source text, that is, the *semantic* structure of the source text. These concepts are visualized in Figure 7.

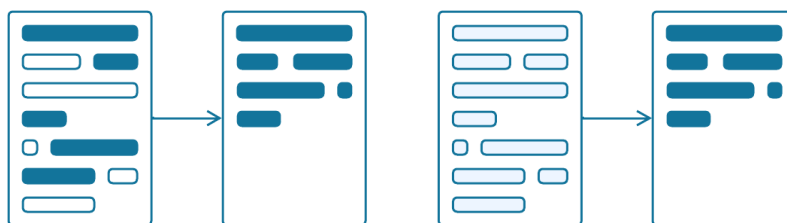


Figure 7: A sketch of extractive (left) and abstractive (right) summarization. The blue parts represent information-dense sentences that are extracted or originally generated for the summary.

Extractive systems include frequency-based methods (identify and prioritize the most frequent words or phrases and use them as a selector for key sentences) and graph-based ranking algorithms (represent text as a graph where nodes are sentences or phrases, and edges represent similarity or semantic relationships; algorithmically rank these sentences by centrality to extract the most representative ones). Abstractive methods include rule-based systems and concept representation and fusion (attempt to identify concepts in the text and combine them in novel ways using structured representations such as semantic graphs). Some systems also implement hybrid approaches, combining elements of both methods, identifying and

extracting key sentences, and then paraphrasing or refining them using abstractive methods.

LLMs excel at generating summaries that meet the requirements mentioned above and that can even surpass human outcomes, although showing limitations such as hallucinations or factual inconsistencies (Pu, Gao, and Wan 2023; Zhang et al. 2023). The summaries created by these models can be classified as abstract, but show extractive tendencies (Raffel et al. 2023). More specifically, they may exhibit a *copying bias* where they replicate answers from the context instead of learning the underlying patterns. This bias can be shown by identifying and pruning neurons that prioritize copying over generalization (Ali, Wolf, and Titov 2024). Furthermore, in contextual information processing, LLMs (even without explicit fine-tuning⁹) often generate answers by copying facts directly from the context, supplemented with additional "glue text" generated by the model, even if these patterns contradict information learned during training. This is also called In-Context Learning (ICL) and it suggests the ability of these models to identify and replicate relevant text from the context even when the information in the target text contradict the "knowledge" in the model's training data (Phukan et al. 2024).

3.2 Interpretability of LLMs

Interpretability in the context of AI-systems refers to the attempt of understanding the internal mechanisms and neural circuits that enable these models to perform specific tasks. There are numerous approaches in pursuit of this goal. Understanding these will shed light on the approaches taken within this work.

Bereska and Gavves (2024) argue that the methods of evaluating AI-systems from the outside-in can be divided into four categories. These can also apply to the interpretation of LLMs.

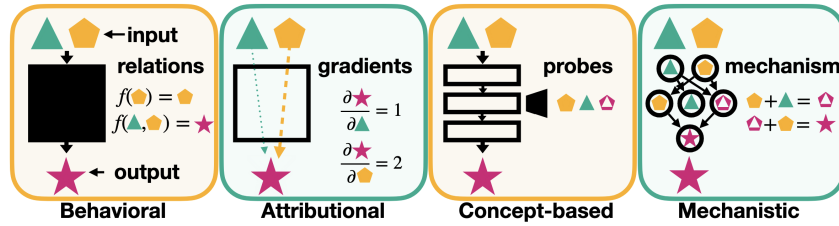


Figure 8: Four methods of interpretability paradigms offering distinct lenses for understanding neural networks (Bereska and Gavves 2024).

Behavioral analyses focus mainly on the relations between model inputs to an ANN and the outputs it produces. Here, the model itself is treated as a black box, and no attempt is made to rigorously understand its internal structure or mechanistic processes. These approaches rely on external evaluation benchmarks such as accuracy¹⁰, BLEU (Papineni et al. 2002) (e.g., for machine translation), or ROUGE (Lin 2004) (e.g. for summary evaluation). To quantify findings, models are usually tested on various inputs under different perturbations to observe robustness and consistency of observed behavior. Behavioral methods are model-agnostic, meaning, that they can be easily applied to proprietary or inaccessible models, but they provide no insights into why and how a model arrives at a certain output.

Attributional approaches attempt to quantify the effects of individual input features (e.g., tokens, pixels) in determining outputs of the model. These methods trace outputs back to specific inputs, often using gradients or layer-wise relevance propagation. Thus, insights into the relevance of specific input components can be provided. To quantify these effects techniques such as SmoothGrad (Smilkov et al. 2017) and Integrated Gradients (Sundararajan, Taly, and Yan 2017) average across gradients. Attributional approaches can clarify decision validation, compliance, and bias detection without requiring an understanding of the model's internal structure. However, discontinuity or sensitivity to small

⁹Fine-tuning refers to the process of training a model on a specific task.

¹⁰Accuracy is calculated by dividing the number of correct predictions given by a model by the total number of predictions.

perturbations may limit the reliability of attributions.

Concept-based methods take a top-down approach as they aim to identify and map high-level, human-understandable representations (i.e., concepts such as gender, syntactic roles, or objects in an image) within the internal activations in an ANN. These approaches try to link abstract concepts to specific parts of a model, similar as in reverse engineering of human-written programs. Used tools are inter alia training supervised auxiliary classifiers (Belinkov 2021), or unsupervised contrastive and structured probes (Burns et al. 2024). Thus, they aim to answer questions such as “Does this group of neurons represent a specific concept like gender or tense?” As mentioned above, this question can often not be definitely answered as each neuron can have different representations depending on the context of the task the model is performing. Due to the complexity of the systems these approaches usually do not uncover the full causal pathway for generating outputs.

Finally, **Mechanistic Interpretability** paradigms analyze the precise internal workings of the neural network in a bottom-up approach. They seek to uncover the causal pathways through which inputs are transformed into outputs. This involves analyzing individual neurons, layers, attention heads, or weight in order to get an intimate view of the model’s inner mechanics. The goal of Mechanistic Interpretability is to get a fundamental, detailed interpretation of a model. This can be achieved by analyzing how specific weights, circuits, and components interact to compute outputs, involving techniques like circuit analysis, attention flow tracing, or activation path analysis. Achieving this goal may enable research to understand, and therefore trust and predict model’s behaviors. However, these approaches are highly detailed and resource-intensive, making them difficult to scale to entire models or datasets.

3.3 Transformers as Compressors

It has been pointed out that Transformers and therefore LLMs represent the algorithmic culture in respect to Breiman (2001) as they map observed inputs to observed outputs without making assumptions about the nature of language itself. Thus, they are merely estimators of their training corpus, compressing its statistical properties into their parameters. This notion highlights the importance of that underlying corpus. It highly depends on the requirements for the Transformer which training corpus is suitable. Veres (2022) argues that the term "LLM" should even be reformed to "*Corpus Model*" since the former would imply that LLMs model natural language itself - a human concept that contains much more than the large amount of texts included in most training corpora. This dependence also refers to language concepts present in the corpus. The summarization process, for instance, can be understood as a mapping from a target text to an abstract, learned from examples in the training data. Estimating this relation precisely on vast amounts of examples enables the model to transfer learn (Laskar, Hoque, and J. Huang 2021) and generalize: new texts that are unknown can be fed through this mapping function to receive an abstracted summary of that text.

Viewing LLMs as compressors of their training corpus enables approaches to try and demystify their behavior by approximating their outputs using statistical data models. N -grams have proven effective in approximating the behavior of LLMs in numerous scenarios. Nguyen (2024) demonstrates that 78% of LLMs next-token distributions on TinyStories (Eldan and Li 2023) align with predictions provided by N -gram rule-sets created from the very same dataset. It stands out that the accuracy of N -gram approximations improves as N grows. That is, because a larger number of preceding symbols gives more meaningful context than a small N . Nguyen (2024) argues that such rules can help detect overfitting¹¹ during training and provide insights into how the models evolve from learning simple to complex statistical rules.

¹¹Overfitting refers to the case in which a model is adjusted too closely to its training data, resulting in a model that precisely predicts the training data but cannot make accurate predictions for test data that is not present in the training data.

Olsson et al. (2022) identify induction heads as a mechanism in Transformers that might enable pattern completion and ICL. These mechanisms implement pattern-matching algorithms to complete sequences like $[A][B]\dots[A] \rightarrow [B]$ ¹² to infer from processed patterns to observed tokens. If LLMs are compressors of their training corpus (as per Veres (2022)), then ICL can be understood as a mechanism that uses this compressed statistical compression in a new way: during training, the model learns to compress statistical patterns from its corpus into its parameters. These patterns include token-level statistics, but also higher-order patterns about how information relates and sequences continue. When a context provides patterns like $[A][B]\dots[A][B]\dots[A] \rightarrow ?$, the model is not taught something entirely new. Instead, statistical patterns are being activated that were already compressed into its parameters during training. The model has estimated that when sequences show repetitive patterns, those patterns tend to continue. This suggests that ICL is not separate from corpus learning - it is a manifestation of it. The model's ability to recognize and continue patterns in the context window depends on having learned similar pattern-recognition mechanisms from its training data. This provides a concrete mechanism for how statistical models in their input can be used to make predictions on their output.

The mechanism explained above also relates to the estimation of stochastic models such as Markov chains. Zekri et al. (2024) demonstrate that a transformer with just constant depth of layers and 1 head per layer is capable of representing k th-order Markov models for arbitrary k , contradicting earlier assumptions that the number of attention heads needs to scale linearly with k . This suggests that transformers may have more compact ways of capturing long-range dependencies than previously understood.

Together, these findings collectively suggest that while Transformers are complex systems, they can be viewed as statistical compressors of their training corpus. They encode statistical patterns from the corpus into their parameters, which enables them to generalize and generate predictions for new, unseen inputs. This perspective highlights the role of the training corpus and explains why the model's behavior can often be approximated by simpler statistical models. The key challenge lies not necessarily in the complexity of individual predictions, but in understanding how Transformers select which statistical patterns to apply in different contexts.

¹²If there is a pattern of $[A]$ being followed by $[B]$ in the context, it is likely that the Transformer will also predict $[B]$ if $[A]$ is the final token in the context.

4 Approach

It has been established that text summarization is a complex task that requires both the compression of information and the preservation of essential meaning. While LLMs excel at this task, it is challenging to evaluate observations on them if the underlying natural data (language) is too complex and the quality of a sequence (e.g., a sentence) is dependent on more than statistical properties. A controlled environment is needed, which can be provided by using Markov chains. A k th-order Markov chain can produce synthetic data whose statistical properties are fully determined by its transition probabilities. The notion that a stochastic model compresses a data-generating process into its parameters can be tested and quantified on the Transformer now: If the Transformer is able to precisely estimate the statistical structure of the synthetic data it is a compressor of this very dataset. Since the data is generated by a well-known model,

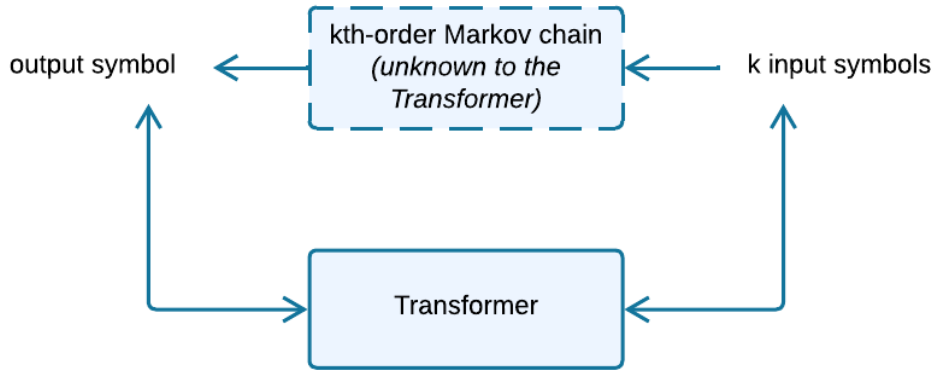


Figure 9: A depiction of how a Transformer can be viewed as a compressor of a known data-generating process - a Markov chain. The Transformer estimates the relationships from input to output without making assumptions about the underlying Markov chain.

it can be precisely measured how well these properties are preserved. To study pattern preservation at different scales, the amount of output tokens of the Transformer can be varied. Naturally, a smaller number of symbols provides less room to represent statistical patterns, especially for higher-order Markov chains.

To begin exploring this notion, the approach taken in this work works as a foundational experiment: training a small Transformer model on binary, Markov-generated sequences. This experimental setup serves as a simplified analog to the summarization task. By observing how the small Transformer learns to capture and reproduce the statistical patterns inherent in Markov data, the aim is to gain insights into the broader question of how LLMs might preserve and regenerate information in summarization tasks. It has already been shown that even small Transformers perform low test loss on sequences generated from these patterns (Rajaraman et al. 2024). Under this perspective, the focus of this approach lies in the model’s output. The subject of investigation is the relationship between sequence length and the model’s ability to capture underlying patterns. In effect, this translates to understanding how the length of a generated summary impacts its ability to retain the statistical structure of the target text. The hypothesis is that there exists an optimal sequence length m of tokens at which the Transformer model can most effectively capture the statistical structure of an underlying Markov chain while minimizing m . This optimal length may vary depending on the complexity of the underlying Markov process (i.e. the order and the entropy of the chain’s stationary distribution), mirroring how the ideal summary length might depend on the complexity and information density of the original text.

While the initial experiment uses simple binary sequences, the principles it explores have far-reaching

implications for language-based summarization tasks. By establishing a quantitative framework based on Markov chains, a goal of this approach is to develop metrics that can assess how well summaries preserve the statistical properties of their source texts. It provides a new perspective on summarization quality, complementing traditional metrics with a rigorous analysis of structural adherence.

The following research questions are raised:

1. **How effectively can Transformers compress and reproduce the statistical properties of Markov chains?**
 - (a) How accurately can the Transformer preserve the original chain's transition probabilities in its generated sequences?
 - (b) How does the length of the output sequence affect the preservation of these statistical properties?
 - (c) What is the minimal sequence length of tokens required to maintain essential statistical patterns for a given Markov order?
2. **How does the complexity of the underlying statistical process affect pattern preservation?**
 - (a) How does increasing the Markov chain order affect the Transformer's ability to preserve statistical patterns?
 - (b) Is there a relationship between the entropy of the chain's stationary distribution and the minimal sequence length needed for effective pattern preservation?

The precise structure of the approach, the metrics, the Transformer model, et cetera are further elaborated in the section below.

5 Experiment Setup

This section presents the detailed experimental design and setup employed to investigate the relationship between sequence length of tokens and the estimation of Markov chains. All experiments were implemented and executed in Python using Jupyter Notebook. The complete source code and experimental notebooks are publicly available at GitHub¹³.

5.1 Intuition and Goal

The hypothesis is that Transformers, when trained on sequences from a Markov chain, act as compressors of the underlying stochastic model. Just as Transformers compress their training corpus into their parameters for natural language tasks, they should be able to compress the statistical properties of a Markov chain and reproduce these patterns in their outputs. This relates to the broader question of how they preserve statistical patterns across different sequence lengths - an important aspect of ICL and text summarization.

The approach taken in this thesis works as a miniature experiment to evaluate Transformers as compressors. By using Markov chains as synthetic data generators, an environment is created where the statistical properties of the generating process are well-known and properties of the underlying model are well-studied, providing theoretical benchmarks against which the Transformer’s performance can be compared. The goal is to quantify how effectively Transformers can compress and reproduce Markov patterns based on the sequence length of the Transformer’s output. The relevant examined parameters are:

1. The order of the Markov chain (1 to 6)
2. The complexity of the Markov chain (measured by the entropy of chain’s stationary distribution)
3. The output length (100 up to 5000)

Additional variables such as context size, temperature in the softmax function, and context length are noted in the analysis but are not further examined in the experiments.

5.2 Metrics

Two metrics have been chosen as an estimate of the difference of the underlying transition matrix (ground truth) and the estimated matrix for the model output.

5.2.1 Kullback-Leibler Divergence

$$D_{KL}(\mathbf{P} \parallel \mathbf{Q}) = \sum_{x \in \mathcal{S}} \mathbf{P}(x) \log \left(\frac{\mathbf{P}(x)}{\mathbf{Q}(x)} \right) \quad (12)$$

The Kullback-Leibler Divergence (KLD) quantifies the information loss when approximating a true probability distribution $\mathbf{P}$ with a predicted distribution $\mathbf{Q}$. In this context, it measures the divergence between the true Markov transition distribution $\mathbf{P}$ and the distribution $\mathbf{Q}$ estimated from the Transformer’s generated sequences. For sequences of length m , some state transitions might not appear in the generated sequence, leading to undefined probabilities in the transition matrix. The implementation in this approach (Listing 2) only considers defined states when computing the KLD.

For a binary Markov chain with a fixed reference distribution $\mathbf{P}$, there exists a distribution $\mathbf{Q}$ that maximizes the KL-divergence:

$$\max_{\mathbf{Q}} D_{KL}(\mathbf{P} \parallel \mathbf{Q}) = \max_{\mathbf{Q}} \sum_i p_i \log \left(\frac{p_i}{q_i} \right) \quad (13)$$

¹³<https://github.com/moritzwosch/markov-transformer>

subject to the constraints:

$$\sum_j q_{ij} = 1 \quad \text{for all states } i, \quad (14)$$

$$q_{ij} \geq \epsilon \quad \text{for all transitions } i, j, \quad (15)$$

where $\epsilon = 10^{-10}$ is a smoothing factor to avoid undefined logarithms. For each row $[p, 1 - p]$ in the reference distribution $\mathbf{P}$, the maximum KLD is achieved when $\mathbf{Q}$ has the corresponding row $[\epsilon, 1 - \epsilon]$ (if $p > 0.5$) or $[1 - \epsilon, \epsilon]$ (if $p < 0.5$).

5.2.2 Mean Absolute Error

The Mean Absolute Error (MAE) is the arithmetic average of the absolute errors between two distributions:

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |\mathbf{P}_i - \mathbf{Q}_i| \quad (16)$$

where $\mathbf{P}_i$ and $\mathbf{Q}_i$ are the corresponding transition probabilities in the true and estimated Markov chains, respectively, and n is the number of defined transitions. Unlike the KLD, which is sensitive to small probability differences when probabilities are close to 0 or 1, the MAE provides a linear measure of difference between the transition probabilities. This makes it particularly useful for assessing the model's ability to capture the overall structure of the transition matrix, regardless of whether the probabilities are balanced or skewed. For a k th-order Markov chain with binary states, the MAE is calculated across all $2^k \times 2$ transition probabilities where transitions are defined in both the true and estimated matrices. This metric complements the KLD by providing a more intuitive measure of how far the estimated probabilities deviate from the true probabilities on average.

5.3 Markov Functions

`generate_transition_matrix` (Listing 4) creates a random transition matrix $\mathbf{M}$ for a k th-order Markov chain of dimension $2^k \times 2$, where each row corresponds to a possible state (represented by the previous k bits) and contains transition probabilities to the next bit (0 or 1). The function ensures that:

$$\sum_{x \in \{0,1\}} p(x|s) = 1 \quad \text{for all states } s, \quad (17)$$

through row-wise normalization. The function `generate_sequence` (Listing 5) implements the stochastic process defined by the transition matrix. Given a k th-order Markov chain with transition matrix $\mathbf{M}$, it generates a sequence $(X_t)_{t=1}^n$ of length n . The process follows:

1. Initialize $X_1, \dots, X_k$ randomly from $\{0, 1\}$.
2. For $t > k$, generate X_t according to $\mathbf{M}$.

The function `estimate_matrix` (Listing 6) estimates the transition probabilities from an observed sequence. For a sequence $(x_t)_{t=1}^n$, it constructs the empirical transition matrix $\hat{\mathbf{M}}$, where each element is computed as a conditional k -gram. To handle unobserved states (where the denominator is zero), the function assigns invalid probabilities $[0.0, 0.0]$ to ensure that these rows are not well-defined.

5.4 Transformer Model

The Transformer models used in this study were designed to balance computational efficiency with the need to capture statistical patterns in Markov-generated sequences. They were constructed according to the Markov order of the underlying chain, including 2-layer, 3-layer, and 4-layer attention architectures, each with one head per layer. The model architectures relate to Vaswani et al. (2017) with a binary alphabet. The parameters were chosen in view of Rajaraman et al. (2024). The models were trained on binary sequences generated from Markov chains of varying orders $k \in \{1, \dots, 6\}$. Sequence lengths in the range of 100-5000 tokens were selected to explore the relationship between output length and pattern preservation. These lengths align with the theoretical requirements for capturing higher-order dependencies in Markov chains, as longer sequences provide more context for estimating statistical transitions. The training parameters were adjusted for each order to accommodate the increasing complexity. The parameters are visible in Table 1.

Table 1: Training parameters for the Transformer models used in the experiments.

Universal Parameters						
Models per order	5					
Attention Heads	1					
Batch size	16					
Learning rate	0.001					
Weight decay	0.001					
Dropout	0.0					
Context size	512					
Optimizer	AdamW					
Parameter	Order of Markov chain					
	1	2	3	4	5	6
Number of Layers	2	2	3	3	3	4
Embedding Dimension	16	64	64	64	64	64
Sequence Length	1000	1000	1000	2000	2000	2000
Number of Sequences	500	500	1000	1000	2000	2000
Training Iterations	5000	5000	10000	10000	15000	15000

5.5 Experiment Pipeline

1. Generate Markov chains of orders 1-6 (5 chains per order)
2. Train Transformer models on sequences from these chains.
3. Generate output sequences of varying lengths (100-5000 tokens).
4. Generate random sequences of varying lengths (100-5000 tokens).
5. Sample sequence that minimizes the KLD.
6. Compare statistical properties of Original Markov chains and estimated chains using both metrics.

The approach is demonstrated in Table 2 and Figure 10.

Table 2: Overview of the experiment pipeline.

Method	Input	Output
Transformer Model	Context sequence (50 tokens)	Generated sequence (100-5000 tokens)
Random Process	None	Random binary sequence (100-5000 tokens)
Lower-Bound-Sampling	Markov chain parameters	Sampled sequence (100-5000 tokens)

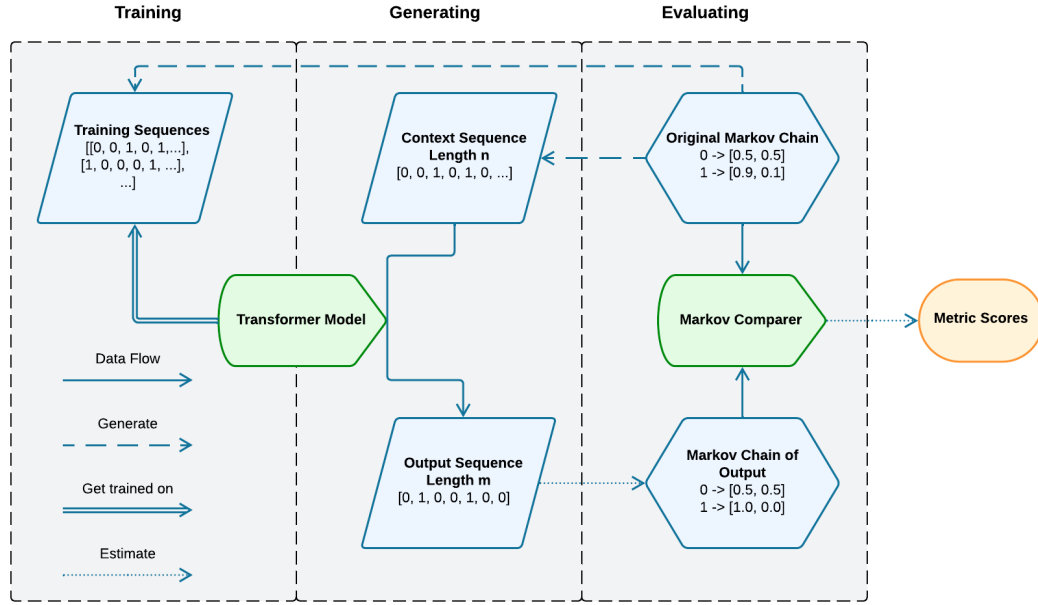


Figure 10: A depiction of the approach with example values for a 1st-order Markov chain. The approach includes training a Transformer model on sequences drawn from a ground-truth Markov chain and having the model predict next tokens for given context sequences. A Markov chain is then estimated for the model's output sequences and compared to the ground-truth.

6 Results and Discussion

This section presents and discusses the results of the experiments, followed by an error-analysis. Furthermore, the stability of the results is verified.

6.1 Training Dynamics

The results show convergence in the training loss values for all models. The mean point of convergence increases with the Markov order of the underlying chains. A quantitative analysis can be seen in Table 11. None of the models showed a sign of under- or overfitting on the training data. The theoretical minimum cross-entropy for a Markov chain can be calculated by converting the $2^k \times 2$ transition matrix into a full state transition matrix ($2^k \times 2^k$) and using the stationary distribution π using power iteration. The entropy rate can be derived using the code in Listing 7. This represents the theoretical lower bound for any model trying to predict the next bit. In relation to this lower bound, the loss curves were plotted, and the achieved losses were compared to this optimum. Figure 17 shows the loss curves for each Transformer model, Figure 11 shows the averaged loss curve for every Markov order, and Table 12 shows the relative achieved loss vs. the optimum per order. It is visible that the Transformer models for different Markov

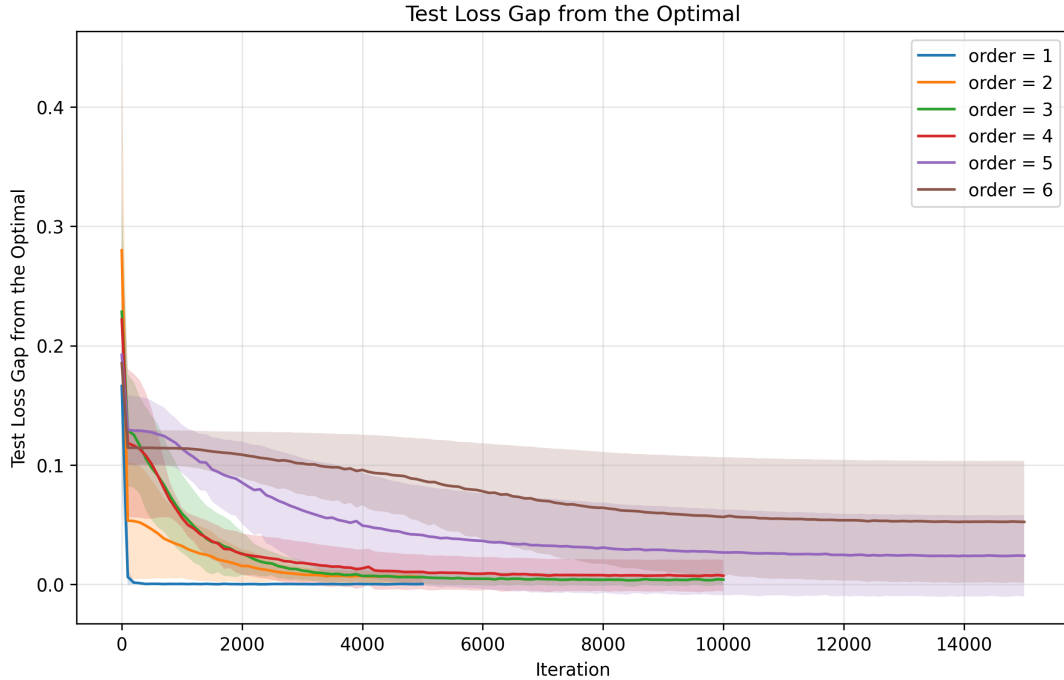


Figure 11: Averaged loss curves for each Markov order in relation to the optimal loss value.

orders show distinct convergence patterns and relationships with their theoretical optima. The 1st-order Transformers demonstrate the most rapid convergence, reaching their optimal values within 100 iterations with minimal deviations. This perfect convergence to the theoretical optimum suggests that the model architecture is well suited for capturing 1st-order dependencies in Markov sequences. For orders 2-4, the models show similar convergence behavior, but they require longer training periods, stabilizing after 4000-5000 iterations. The fact that the Transformers for Markov order 2 show slightly deviant behavior can be explained by the fact that the Markov chains for order 2 have comparably lower entropy values for their stationary distributions which leads to a lower optimal loss in general. This effect could be compensated for with a higher number of models for different chains, reducing the variance. Higher-order models (orders 5-6) display notably different behavior. They exhibit slower initial convergence rates,

extended training requirements (8000-10000 iterations), larger final gaps from theoretical optima (1.044x and 1.091x, respectively), and higher variance in their learning trajectories.

6.2 Output Performance and Error Distribution

Both the order of the ground-truth Markov chain’s order and the length of the output sequence appear to have a strong effect on the model performance in estimating the underlying Markov chain. The metrics also play a crucial role in relation to these two parameters.

Two line charts that compare the output length vs. KLD/MAE across different Markov orders can be seen in Figure ???. The associated heatmaps can be seen in Figure ???.

For every Markov order threshold sequence lengths were calculated, at which the Transformer models reached a stationary distribution and the metric scores did not improve for longer sequences. The mean metric scores μ and the standard deviation σ at this point were calculated as:

$$\mu = \frac{1}{N} \sum_{i=1}^N x_i \quad \text{and} \quad \sigma = \sqrt{\frac{1}{N-1} \sum_{i=1}^N (x_i - \mu)^2} \quad (18)$$

The results are presented in Table 3. The relationship between Markov order and model performance with

Table 3: Average threshold sequence lengths and performance metrics by Markov order.

Order	Threshold Length*	KLD at Threshold	MAE at Threshold
1	5000 (5000)	0.001 $\pm$ 0.001	0.006 $\pm$ 0.004
2	5000 (3000)	0.894 $\pm$ 2.088	0.128 $\pm$ 0.055
3	400 (4000)	0.154 $\pm$ 0.286	0.056 $\pm$ 0.015
4	4000 (5000)	0.203 $\pm$ 0.302	0.045 $\pm$ 0.039
5	2000 (5000)	3.068 $\pm$ 2.655	0.132 $\pm$ 0.043
6	5000 (5000)	5.158 $\pm$ 3.632	0.135 $\pm$ 0.071

*Threshold Length: Sequence length at which performance stabilizes (additional threshold length for the lower bound sequences)

respect to model output shows distinct patterns in both metrics. The analysis of the results reveals several key findings: The analysis shows that the KLD increases exponentially with the Markov order. For a fixed sequence length, higher-order models consistently show larger KLD values, with the relationship following an exponential pattern (except for the outlier order 2 which demonstrates a higher score respectively). This suggests that the complexity and difficulty of accurately modeling the sequences grows exponentially with the order of the Markov chain.

The effect of sequence length on model performance demonstrates consistent patterns across all Markov orders: The relationship between sequence length and error metrics (KLD and MAE) shows a clear exponential decay pattern for every order. As the sequence length increases, both error metrics decrease rapidly at first and then stabilize, following a characteristic curve similar to exponential decay. This pattern is consistent across all Markov orders (except order 2), though the absolute values and rate of improvement vary with order. Higher-order models typically require longer sequences to achieve stable performance. However, the threshold sequence length from which convergence is achieved shows no clear correlation to the Markov order: For orders 1,4,5, and 6 the metrics converge not before 5000 tokens while they converge earlier for orders 2 and 3. The error distribution is shown in Figure 12.

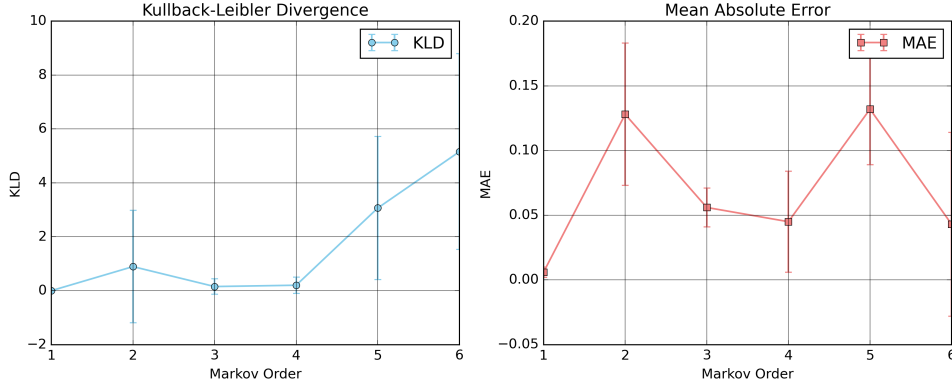


Figure 12: Error distribution across different Markov chains.

6.3 Comparative Evaluation

The outputs of the Transformer models were put into relation with a baseline (random walk) process and a lower bound that was sampled to minimize the KLD. Figure 13 shows this comparison across all Markov orders.

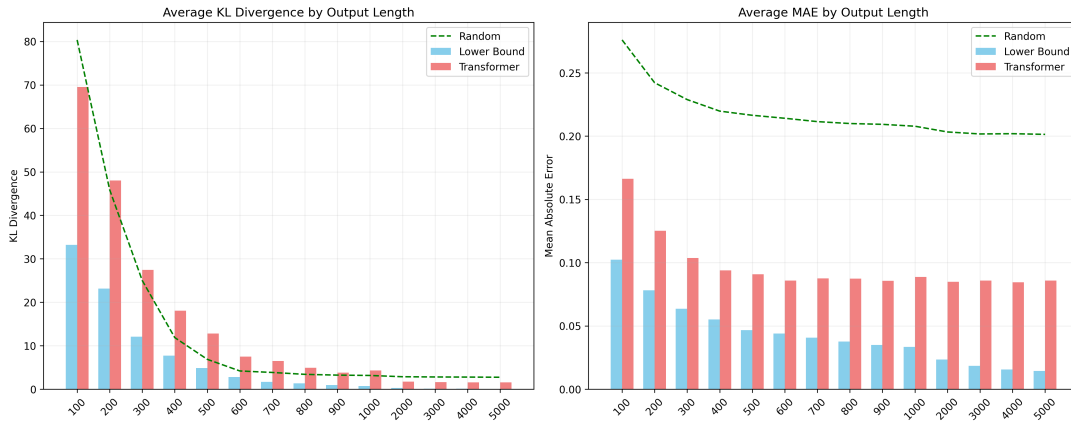


Figure 13: Sequence length vs. the averaged KLD and MAE (over all orders) for the Transformers, the lower bound, and the random walk.

6.3.1 Random vs. Transformer Performance

The results show that the Transformer outperforms the random baseline for every length regarding the MAE and for sequences with $m \geq 2000$ for the KLD. The random baseline stabilizes after the same output length at ≈ 2.5 (KLD) and at ≈ 0.20 (MAE) across all orders. The fact that the random walk outperforms the Transformer model for higher orders in the KLD can be attributed to the random walk's ability to achieve balanced transitions across all states, even with shorter sequence lengths. In contrast, the Transformer model demonstrates high deviations for certain transitions, particularly those that are rarely visited in its output. Because the KLD is highly sensitive to such deviations, this explains the advantage of random walks in this context. However, as the sequence length increases, the Transformer's abilities to capture the underlying statistical patterns become evident, outperforming the random walk and demonstrating that it is far better than random guessing for longer sequences.

6.3.2 Lower Bound vs. Transformer Performance

For 1st-order Markov chains, the Transformer arrives at exceptional performance with $\text{KLD} = 0.01 \pm 0.01$, nearly matching the lower bound score. The model maintains a consistently low distance to the lower bound (≤ 0.05) for both metrics across all sequence lengths. 2nd-order chains (see Fig 16) show increased

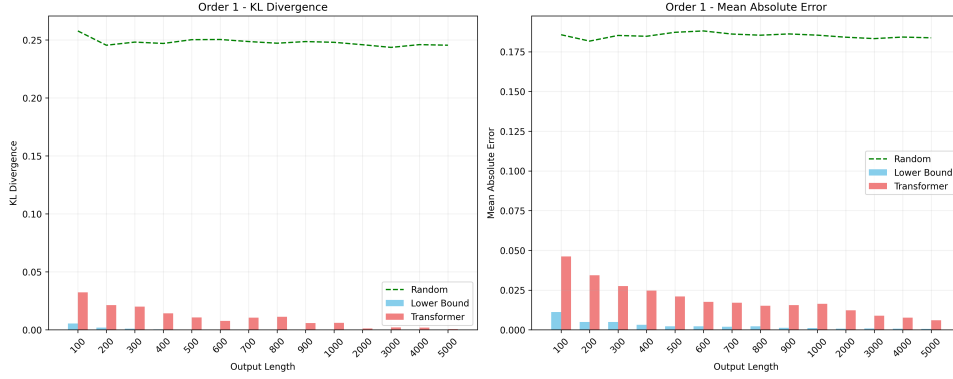


Figure 14: Markov order 1: Sequence length vs. the averaged KLD and MAE for the Transformers, the lower bound, and the random walk.

variance and do not show clear convergence. In fact, for both metrics the performance decreases above 2000 tokens output length while the lower bound shows clear convergence.

Moving to medium-order chains (see Figures 20–21), the 3rd-order model converges relatively quickly at 400 tokens, but shows an increased deviation from the lower bound (≥ 0.2 for the KLD and ≥ 0.05 for the MAE). The 4th-order model, while requiring longer sequences (≈ 4000 tokens) converges closer to the lower bound (≥ 0.05 for both metrics).

High-order chains (orders 5-6) demonstrate substantially different behavior and converge at a clearly higher point for both metrics compared to the lower bound. This analysis suggests that, while the

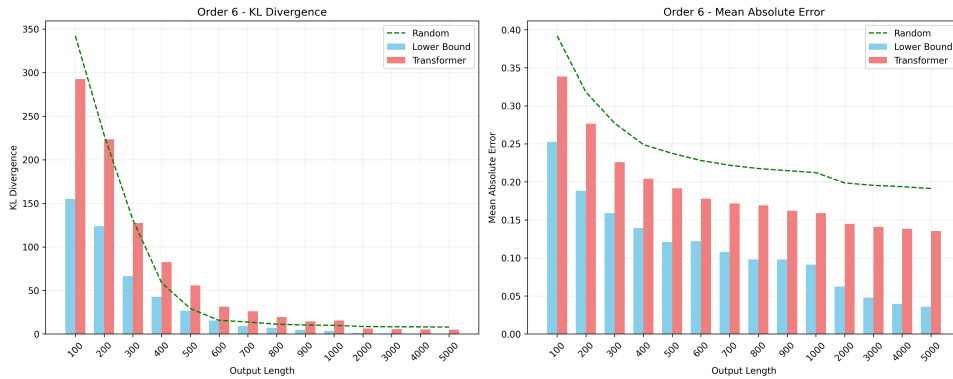


Figure 15: Markov order 6: Sequence length vs. the averaged KLD and MAE for the Transformers, the lower bound, and the random walk.

Transformer architecture excels at capturing lower-order dependencies, it faces fundamental challenges with higher-order patterns, requiring longer sequences and showing increased variance in performance. The trade-off between model complexity and achievable performance becomes particularly clear beyond order 4.

6.4 Statistical Validation

To validate the reliability and significance of the results, several statistical approaches were employed. The validation process focused on three main aspects: consistency across different random seeds, sufficient sample size, confidence intervals for performance metrics, and statistical significance of performance differences between orders.

6.4.1 Performance Stability

Table 4: Statistical analysis of Transformer’s performances across test runs.

Order	KLD	MAE	N
	Mean $\pm$ Std [95% CI]	Mean $\pm$ Std [95% CI]	
1	0.01 ± 0.021 [0.009, 0.012]	0.019 ± 0.018 [0.018, 0.021]	700
2	1.322 ± 3.582 [1.056, 1.588]	0.09 ± 0.056 [0.086, 0.094]	700
3	0.612 ± 2.249 [0.445, 0.779]	0.064 ± 0.043 [0.061, 0.068]	700
4	3.494 ± 8.25 [2.882, 4.107]	0.077 ± 0.045 [0.074, 0.08]	700
5	19.213 ± 26.727 [17.228, 21.198]	0.143 ± 0.046 [0.139, 0.146]	700
6	65.063 ± 89.822 [58.392, 71.733]	0.188 ± 0.077 [0.182, 0.194]	700

N = Number of test runs

The analysis in Table 4 reveals several significant patterns. For 1st-order Markov chains, the model demonstrates exceptional stability and accuracy, with a mean KLD of 0.01 ± 0.021 and narrow confidence intervals [0.009, 0.012]. The corresponding MAE values (0.019 ± 0.018) show similar characteristics. A notable increase in variance is observed as the Markov order increases. 2nd-order chains show significantly higher variation in KLD (1.322 ± 3.582), while the MAE remains relatively stable (0.09 ± 0.056). Interestingly, 3rd-order models show better performance than 2nd-order in terms of both metrics. The most dramatic changes in stability occur in higher-order models. 5th-order chains show a substantial increase in KLD (19.213 ± 26.727), while 6th-order models demonstrate the highest variance (65.063 ± 89.822). This pattern is mirrored in the MAE values, which increase from 0.143 ± 0.046 (order 5) to 0.188 ± 0.077 (order 6).

The large sample size ($N = 700$) for each order provides strong statistical confidence in these results. The 95% confidence intervals consistently show minimal overlap between orders, indicating statistically significant differences in performance across orders. The widening of these intervals with increasing order (from [0.009, 0.012] for order 1 to [58.392, 71.733] for order 6) quantifies the decreasing stability of model performance at higher orders.

These findings statistically validate earlier observations on the relationship between Markov order and model performance, demonstrating that the degradation in performance with increasing order is not only substantial, but also statistically significant.

6.5 Error Analysis

Various errors that are visible in the results remain open to analysis. This subsection will provide possible answers for any anomalies and outliers in the results.

6.5.1 Performance for Order 2

The 2nd-order Markov chains in the experiments show an interesting pattern that distinguishes them from other orders. Although the Transformer model achieves good training performance for these chains, with a final loss of close to the optimum with a near-optimal difference factor of only 1.015x, the output

behavior shows significant deviation compared to the other orders. Furthermore, the 2nd-order scores do not converge after a certain sequence length. Instead, the performance decreases for sequences above 1000 tokens.

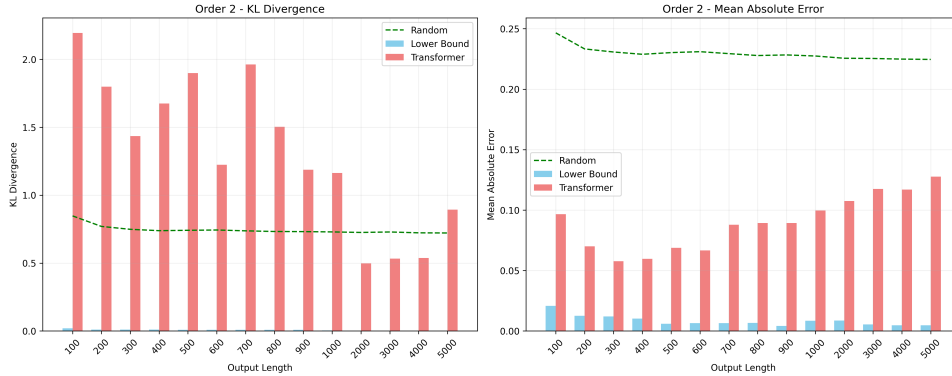


Figure 16: Markov order 2: Sequence length vs. the averaged KLD and MAE for the Transformers, the lower bound, and the random walk.

Examination of the transition matrices in Table 6 reveals highly skewed probability distributions. For example, chain 2.3 exhibits transition probabilities of 0.998/0.002 for state "00" and chain 2.4 shows probabilities of 0.906/0.094 for state "11". These extreme probabilities create unbalanced stationary distributions, which might have an effect on the model's output generation in relation to the KLD.

The discrepancy between training and output performance may be especially pronounced in the KLD measurements due to two factors: First, the KLD is more sensitive to extreme deviations when comparing empirical with actual distributions. This is especially visible when dealing with highly skewed probability distributions, as found in the 2nd-order chains.

Secondly, the training sequences for such a chain are likely to show insufficient occurrences for states with low values in the stationary distribution. The Transformer might not estimate the transition probabilities for such states accurately, and when confronted with them in the output sequences (which becomes more likely for longer sequences) it might produce outputs that do not represent the ground-truth chain.

Interestingly, this anomaly is less visible for higher-order chains, despite their increased complexity. For example, the 3rd-order chains achieve a lower KLD (0.154 ± 0.286) and MAE (0.056 ± 0.015) at their threshold sequence length (400 tokens). This suggests that the anomaly is caused by the specific 2nd-order chains and may be compensated with a larger number of examples. This is actually visible in the results for Chain 2.2 which is defined by more balanced transition probabilities. To address the anomaly, the following strategies could be explored in future work:

- **Data Augmentation:** Increasing the length of training sequences or the number of total training sequences to ensure adequate representation of low-probability transitions.
- **Alternative Metrics:** Using metrics less sensitive to extreme probabilities, such as Jensen-Shannon Divergence (JSD).

6.5.2 Performance for Orders 5 and 6

The Transformer models for higher-order Markov chains (orders 5 and 6) show significant degradation in pattern preservation compared to lower-order chains. This effect is visible for both metrics, as shown in Table 3. The KLD values for orders 5 and 6 are 3.068 ± 2.655 and 5.158 ± 3.632 , respectively, which are substantially higher than those for lower orders. Similarly, the scores for the MAE are 0.132 ± 0.043 and 0.135 ± 0.071 , indicating a consistent increase in error with a growing Markov order. This observed

degradation can be attributed to the exponential growth in the complexity of Markov chains by order. For a k -th order Markov chain with a binary state space, the number of unique states in the transition matrix is 2^k . Let n be the length of the training sequences for the Transformers. To accurately estimate transition probabilities, n must be large enough to provide adequate coverage of all possible transitions. The expected number of occurrences of each state in the training data can be approximated as:

$$E[\text{occurrences}] = \frac{n}{2^k}.$$

For reliable estimation, it is generally desirable that $E[\text{occurrences}] \geq 100$, which implies:

$$n \geq 100 \cdot 2^k.$$

For example, for a 5th-order Markov chain, the number of states is $2^5 = 32$, and for a 6th-order chain the number of states is $2^6 = 64$. Assuming that each state must occur at least 100 times on average for an accurate estimation, the minimum required sequence lengths would be approximately $n = 3200$ for $k = 5$ and $n = 6400$ for $k = 6$. However, this approximation assumes a uniform distribution of states, which may not hold in practice. Skewed stationary distributions or imbalanced transition probabilities can result in some states being underrepresented, further increasing the required sequence length for accurate estimation.

Furthermore, the results indicate that the output sequence lengths used in this approach (up to 5000 tokens) are insufficient to preserve the statistical properties of higher-order chains. Even for the lower bound the curves do not converge at around 0 for both metrics. The performance gap between the Transformer models and the theoretical lower bound increases with the Markov order. For the 6th-order chains, the KLD of the Transformer model is approximately 10 times higher than the lower bound. Interestingly, the random baseline achieves a relatively stable KLD of approximately 2.5 across all orders, suggesting that the Transformer struggles to outperform even a random process for higher-order chains. To improve the performance of Transformer models on higher-order Markov chains, the following strategies could be explored:

- **Increasing Training Sequence Length:** Extending the training sequence length beyond 5000 tokens would provide better coverage of the state space, reducing sparsity and improving the accuracy of transition probability estimation.
- **Data Augmentation:** Generating additional synthetic sequences to supplement the training data could enhance the estimation performance of the Transformers.
- **Alternative Architectures:** Exploring modifications to the Transformer architecture, such as increasing the number of attention heads or layers, could enhance the model's capacity to capture complex dependencies in higher-order chains.

7 Conclusion

Together, these findings suggest that small Transformer models are capable of compressing Markov chains from order 1 to 6 into their parameters and create outputs that represent these underlying chains. The specific contributions of this thesis and how the methodology relates to the broader subject of text summarization are elaborated in this section.

7.1 Contributions

This thesis makes several contributions to understanding the pattern preservation capabilities of Transformer models and their application as statistical compressors. The main contributions are as follows.

1. **Framework for Statistical Pattern Analysis:** The work used a systematic framework to evaluate the abilities of Transformers to preserve statistical patterns using Markov chains as controlled data generators. This framework provides a straightforward methodology for quantifying pattern preservation through two metrics (KLD and MAE), statistical validation methods with robust confidence intervals, and baseline comparisons with random sequences and theoretical lower bounds.
2. **Quantitative Understanding of Sequence Length Impact:** A clear relationship between sequence length and pattern preservation was established which identified critical threshold lengths for performance stabilization across different Markov orders. Furthermore, this quantified the relationship between the Markov chains complexity (measured by the entropy of the chain's stationary distribution) and the required sequence length.
3. **Statistical Compression Capabilities:** The thesis provides empirical evidence of Transformers functioning as statistical compressors: The models achieved near-optimal performance for 1st-order chains, maintained stable performance through fourth-order chains, and demonstrated meaningful pattern preservation even in higher-order chains (5th and 6th order). Increased deviation for higher-order chains can be explained by the length of the training sequences.

This work establishes a foundation for studying summarization through a statistical lens by creating a framework for evaluating information retention across varying sequence lengths. It further demonstrates the connection between statistical compression and information preservation and thus provides methodological tools for studying complex linguistic pattern preservation. The findings have implications for both theoretical understanding of Transformer architectures and practical applications in sequence modeling and summarization tasks.

7.2 Future Work

The broader goal in this research aims to understand the summarization process through LLMs by viewing it through the lens of statistical compression. The notion of Transformers as data compressors directly relates to how they might compress a text into a summary. This work proposes modeling the summarization process using inhomogeneous Markov chains, where each hierarchical group of a document (e.g. paragraphs or sentences) is represented as an individual Markov sub-chain. This idea is illustrated in Figure 23. In this framework, transitions exist not only between words within a group but also between the sub-chains representing different hierarchical levels. For example, while summarizing chapter 1, there exists a certain probability for transitioning to summarizing chapter 2, and this probability increases with time as more words are generated for chapter 1's summary. This stochastic approach reflects the natural flow of summarization, viewing it as a process of creating a shorter version that preserves essential statistical patterns from the original text. A key insight is that information dense subsections containing

critical content would have higher probabilities of being included in summary, while less dense sections might be compressed more significantly or skipped. This aligns with the way LLMs appear to identify and preserve important information when generating summaries. The view that syntactic information is inherently encoded within the stochastic structure of Markov chains supports the hypothesis that LLMs internalize and manipulate symbolic systems embedded in language. This enables them to perform complex tasks like summarization by preserving both statistical patterns and meaningful structure. The experimental results with simple Markov chains provide a foundation for understanding how LLMs might compress and preserve patterns at different scales, suggesting mechanisms for how they maintain coherence and meaning in summarization tasks.

List of Figures

1	An example for a 1st-order Markov chain, where the generated text is represented by the labels on the nodes.	17
2	An illustration of a 2nd-order Markov chain that is estimated for Shakespeare's Hamlet. The rows show all possible sequences of size $k = 2$ while the columns represent all existing words in the document. The cells show how likely the word in the column follows the context in the row.	17
3	An example for a Phrase Structure Grammar and a sentence produced by it (Veres 2022).	18
4	Sentences generated from four word-level N -grams computed from a collection of Shakespeare's sonnets. All characters were mapped to lower-case and symbols were treated as independent words. The sentences were created using Listing 1.	19
5	The original Transformer model architecture (Vaswani et al. 2017).	21
6	The Multi-Head Attention Mechanism (Vaswani et al. 2017).	22
7	A sketch of extractive (left) and abstractive (right) summarization. The blue parts represent information-dense sentences that are extracted or originally generated for the summary.	24
8	Four methods of interpretability paradigms offering distinct lenses for understanding neural networks (Bereska and Gavves 2024).	25
9	A depiction of how a Transformer can be viewed as a compressor of a known data-generating process - a Markov chain. The Transformer estimates the relationships from input to output without making assumptions about the underlying Markov chain.	28
10	A depiction of the approach with example values for a 1st-order Markov chain. The approach includes training a Transformer model on sequences drawn from a ground-truth Markov chain and having the model predict next tokens for given context sequences. A Markov chain is then estimated for the model's output sequences and compared to the ground-truth.	33
11	Averaged loss curves for each Markov order in relation to the optimal loss value.	34
12	Error distribution across different Markov chains.	36
13	Sequence length vs. the averaged KLD and MAE (over all orders) for the Transformers, the lower bound, and the random walk.	36
14	Markov order 1: Sequence length vs. the averaged KLD and MAE for the Transformers, the lower bound, and the random walk.	37
15	Markov order 6: Sequence length vs. the averaged KLD and MAE for the Transformers, the lower bound, and the random walk.	37
16	Markov order 2: Sequence length vs. the averaged KLD and MAE for the Transformers, the lower bound, and the random walk.	39
17	The loss curves for each Transformer model in relation to the optimum value. The associated transition matrices for each Markov chain can be seen in Tables 5–10.	52
18	A Heatmap showing sequence length vs. metric score across all orders for both metrics.	53
19	A heatmap showing Markov order vs. sequence length vs. metric score for both metrics.	53
20	Markov order 3: Sequence length vs. the averaged KLD and MAE for the Transformers, the lower bound, and the random walk.	53
21	Markov order 4: Sequence length vs. the averaged KLD and MAE for the Transformers, the lower bound, and the random walk.	53
22	Markov order 5: Sequence length vs. the averaged KLD and MAE for the Transformers, the lower bound, and the random walk.	54
23	An illustration of a documents hierarchical structure that is transferred to the summary.	54

List of Tables

1	Training parameters for the Transformer models used in the experiments.	32
2	Overview of the experiment pipeline.	33
3	Average threshold sequence lengths and performance metrics by Markov order.	35
4	Statistical analysis of Transformer's performances across test runs.	38
5	1st-order Markov chains: Transition probabilities for all seeds.	49
6	2nd-order Markov chains: Transition probabilities for all seeds.	49
7	3rd-order Markov chains: Transition probabilities for all seeds.	49
8	4th-order Markov chains: Transition probabilities for all seeds.	49
9	5th-order Markov chains: Transition probabilities for all seeds.	50
10	6th-order Markov chains: Transition probabilities for all seeds.	51
11	Training convergence analysis of the Transformers by Markov order.	52
12	Comparison of achieved vs. optimal loss by Markov order.	52

Listings

1	A Python code to estimate N-Grams and produce text.	55
2	A Python code to calculate the Kullback-Leibler Divergence between two probability distributions.	55
3	A Python code to calculate the Mean Absolute Error between two probability distributions.	56
4	A Python code to generate Markov transition matrices.	57
5	A Python code to generate sequences drawn from a kth-order Markov chain.	57
6	A Python code to estimate a kth-order Markov chain for a given sequence.	58
7	A Python code to calculate the optimal cross-entropy loss for a given kth-order Markov chain.	58

References

- Ali, Ameen, Lior Wolf, and Ivan Titov (2024). *Mitigating Copy Bias in In-Context Learning through Neuron Pruning*. arXiv: 2410.01288 [cs.CL]. URL: <https://arxiv.org/abs/2410.01288>.
- Allahyari, Mehdi et al. (2017). *Text Summarization Techniques: A Brief Survey*. arXiv: 1707.02268 [cs.CL]. URL: <https://arxiv.org/abs/1707.02268>.
- Bahdanau, Dzmitry, Kyunghyun Cho, and Yoshua Bengio (2015). *Neural Machine Translation by Jointly Learning to Align and Translate*. arXiv: 1409.0473 [cs.CL]. URL: <https://arxiv.org/abs/1409.0473>.
- Belinkov, Yonatan (2021). *Probing Classifiers: Promises, Shortcomings, and Advances*. arXiv: 2102.12452 [cs.CL]. URL: <https://arxiv.org/abs/2102.12452>.
- Bengio, Yoshua et al. (2003). “A neural probabilistic language model”. In: *J. Mach. Learn. Res.* 3.null, pp. 1137–1155. ISSN: 1532-4435.
- Bereska, Leonard and Efstratios Gavves (2024). *Mechanistic Interpretability for AI Safety – A Review*. arXiv: 2404.14082 [cs.AI]. URL: <https://arxiv.org/abs/2404.14082>.
- Breiman, Leo (2001). “Statistical Modeling: The Two Cultures”. In: *Statistical Science* 16.
- Burns, Collin et al. (2024). *Discovering Latent Knowledge in Language Models Without Supervision*. arXiv: 2212.03827 [cs.CL]. URL: <https://arxiv.org/abs/2212.03827>.
- Chomsky, Noam (1957). *Syntactic Structures*. The Hague: Mouton and Co.
- Devlin, Jacob et al. (2019). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. arXiv: 1810.04805 [cs.CL]. URL: <https://arxiv.org/abs/1810.04805>.
- Eldan, Ronen and Yuanzhi Li (2023). *TinyStories: How Small Can Language Models Be and Still Speak Coherent English?* arXiv: 2305.07759 [cs.CL]. URL: <https://arxiv.org/abs/2305.07759>.
- Frick, F. and W. H. Sumby (1952). “Control Tower Language”. In: *Journal of the Acoustical Society of America* 24, pp. 595–596. DOI: 10.1121/1.1906939.
- Geva, Mor et al. (2021). *Transformer Feed-Forward Layers Are Key-Value Memories*. arXiv: 2012.14913 [cs.CL]. URL: <https://arxiv.org/abs/2012.14913>.
- Hochreiter, Sepp and Jürgen Schmidhuber (1997). “Long Short-Term Memory”. In: *Neural Comput.* 9.8, pp. 1735–1780. ISSN: 0899-7667. DOI: 10.1162/neco.1997.9.8.1735. URL: <https://doi.org/10.1162/neco.1997.9.8.1735>.
- Hutchins, W. John (2004). “The Georgetown-IBM experiment demonstrated in January 1954”. In: *Proceedings of the 6th Conference of the Association for Machine Translation in the Americas: Technical Papers*. Ed. by Robert E. Frederking and Kathryn B. Taylor. Washington, USA: Springer, pp. 102–114.
- Jiang, Juyong et al. (2024). *A Survey on Large Language Models for Code Generation*. arXiv: 2406.00515 [cs.CL]. URL: <https://arxiv.org/abs/2406.00515>.

- Kaplan, Jared et al. (2020). *Scaling Laws for Neural Language Models*. arXiv: 2001.08361 [cs.LG]. URL: <https://arxiv.org/abs/2001.08361>.
- Laskar, Md Tahmid Rahman, Enamul Hoque, and J. Huang (2021). “Domain Adaptation with Pre-trained Transformers for Query-Focused Abstractive Text Summarization”. In: *Computational Linguistics* 48, pp. 279–320. DOI: 10.1162/coli_a_00434.
- Lin, Chin-Yew (2004). “ROUGE: A Package for Automatic Evaluation of summaries”. In.
- McCulloch, Warren S and Walter Pitts (1943). “A logical calculus of the ideas immanent in nervous activity”. In: *The bulletin of mathematical biophysics* 5.4.
- Nguyen, Timothy (2024). *Understanding Transformers via N-gram Statistics*. arXiv: 2407.12034 [cs.CL]. URL: <https://arxiv.org/abs/2407.12034>.
- Olsson, Catherine et al. (2022). *In-context Learning and Induction Heads*. arXiv: 2209.11895 [cs.LG]. URL: <https://arxiv.org/abs/2209.11895>.
- Papineni, Kishore et al. (2002). “BLEU: a method for automatic evaluation of machine translation”. In: *Proceedings of the 40th Annual Meeting on Association for Computational Linguistics*. Association for Computational Linguistics, pp. 311–318. DOI: 10.3115/1073083.1073135. URL: <https://doi.org/10.3115/1073083.1073135>.
- Petrie, Leonard E. Baum. Ted (1966). “Statistical Inference for Probabilistic Functions of Finite State Markov Chains”. In: *Ann. Math. Statist.*
- Phukan, Anirudh et al. (2024). “Peering into the Mind of Language Models: An Approach for Attribution in Contextual Question Answering”. In: pp. 11481–11495. DOI: 10.48550/arXiv.2405.17980.
- Pu, Xiao, Mingqi Gao, and Xiaojun Wan (2023). *Summarization is (Almost) Dead*. arXiv: 2309.09558 [cs.CL]. URL: <https://arxiv.org/abs/2309.09558>.
- Radford, Alec et al. (2019). “Language Models are Unsupervised Multitask Learners”. In: URL: <https://api.semanticscholar.org/CorpusID:160025533>.
- Raffel, Colin et al. (2023). *Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer*. arXiv: 1910.10683 [cs.LG]. URL: <https://arxiv.org/abs/1910.10683>.
- Rajaraman, Nived et al. (2024). *Transformers on Markov Data: Constant Depth Suffices*. arXiv: 2407.17686 [cs.LG]. URL: <https://arxiv.org/abs/2407.17686>.
- Robine, Jan et al. (2023). *Transformer-based World Models Are Happy With 100k Interactions*. arXiv: 2303.07109 [cs.LG]. URL: <https://arxiv.org/abs/2303.07109>.
- Saussure, Ferdinand de (1916). *Cours de linguistique générale*. McGraw-Hill.
- Seabrook, Eddie and Laurenz Wiskott (2023). “A Tutorial on the Spectral Theory of Markov Chains”. In: *Neural Computation* 35.11, pp. 1713–1796. ISSN: 1530-888X. DOI: 10.1162/neco_a_01611. URL: http://dx.doi.org/10.1162/neco_a_01611.
- Smilkov, Daniel et al. (2017). *SmoothGrad: removing noise by adding noise*. arXiv: 1706.03825 [cs.LG]. URL: <https://arxiv.org/abs/1706.03825>.

-
- Sun, Chi, Luyao Huang, and Xipeng Qiu (2019). *Utilizing BERT for Aspect-Based Sentiment Analysis via Constructing Auxiliary Sentence*. arXiv: 1903.09588 [cs.CL]. URL: <https://arxiv.org/abs/1903.09588>.
- Sundararajan, Mukund, Ankur Taly, and Qiqi Yan (2017). *Axiomatic Attribution for Deep Networks*. arXiv: 1703.01365 [cs.LG]. URL: <https://arxiv.org/abs/1703.01365>.
- Vaswani, Ashish et al. (2017). *Attention Is All You Need*. arXiv: 1706.03762 [cs.CL]. URL: <https://arxiv.org/abs/1706.03762>.
- Veres, Csaba (2022). *Large Language Models are not Models of Natural Language: they are Corpus Models*. arXiv: 2112.07055 [cs.CL]. URL: <https://arxiv.org/abs/2112.07055>.
- Wason, R. (2018). “Deep learning: Evolution and expansion”. In: *Cognitive Systems Research* 52, pp. 701–708. DOI: 10.1016/j.cogsys.2018.08.023.
- Weizenbaum, Joseph (1966). “ELIZA—a computer program for the study of natural language communication between man and machine”. In: *Commun. ACM* 9.1, pp. 36–45. ISSN: 0001-0782. DOI: 10.1145/365153.365168. URL: <https://doi.org/10.1145/365153.365168>.
- Zekri, Oussama et al. (2024). *Large Language Models as Markov Chains*. arXiv: 2410.02724 [stat.ML]. URL: <https://arxiv.org/abs/2410.02724>.
- Zhang, Tianyi et al. (2023). *Benchmarking Large Language Models for News Summarization*. arXiv: 2301.13848 [cs.CL].
- Zhao, Haiyan et al. (2023). *Explainability for Large Language Models: A Survey*. arXiv: 2309.01029 [cs.CL]. URL: <https://arxiv.org/abs/2309.01029>.

A Tables

Table 5: 1st-order Markov chains: Transition probabilities for all seeds.

Context	Chain 1.1		Chain 1.2		Chain 1.3		Chain 1.4		Chain 1.5	
	0	1	0	1	0	1	0	1	0	1
0	0.004	0.996	0.527	0.473	0.764	0.236	0.200	0.800	0.416	0.584
1	0.317	0.683	0.667	0.333	0.462	0.538	0.355	0.645	0.371	0.629

Table 6: 2nd-order Markov chains: Transition probabilities for all seeds.

Context	Chain 2.1		Chain 2.2		Chain 2.3		Chain 2.4		Chain 2.5	
	0	1	0	1	0	1	0	1	0	1
00	0.442	0.558	0.429	0.571	0.998	0.002	0.803	0.197	0.593	0.407
01	0.812	0.188	0.335	0.665	0.337	0.663	0.493	0.507	0.689	0.311
10	0.981	0.019	0.404	0.596	0.920	0.080	0.502	0.498	0.179	0.821
11	0.928	0.072	0.235	0.765	0.496	0.504	0.094	0.906	0.318	0.682

Table 7: 3rd-order Markov chains: Transition probabilities for all seeds.

Context	Chain 3.1		Chain 3.2		Chain 3.3		Chain 3.4		Chain 3.5	
	0	1	0	1	0	1	0	1	0	1
000	0.415	0.585	0.820	0.180	0.096	0.904	0.120	0.880	0.305	0.695
001	0.292	0.708	0.676	0.324	0.492	0.508	0.352	0.648	0.093	0.907
010	0.358	0.642	0.268	0.732	0.262	0.738	0.473	0.527	0.940	0.060
011	0.540	0.460	0.168	0.532	0.099	0.901	0.860	0.140	0.728	0.272
100	0.849	0.151	0.329	0.671	0.638	0.362	0.638	0.362	0.370	0.630
101	0.570	0.430	0.705	0.295	0.799	0.201	0.142	0.858	0.870	0.130
110	0.510	0.490	0.687	0.313	0.176	0.824	0.475	0.525	0.437	0.563
111	0.648	0.352	0.037	0.963	0.645	0.355	0.317	0.683	0.841	0.159

Table 8: 4th-order Markov chains: Transition probabilities for all seeds.

Context	Chain 4.1		Chain 4.2		Chain 4.3		Chain 4.4		Chain 4.5	
	0	1	0	1	0	1	0	1	0	1
0000	0.008	0.992	0.654	0.346	0.511	0.489	0.924	0.076	0.192	0.808
0001	0.110	0.890	0.728	0.272	0.550	0.450	0.364	0.636	0.246	0.754
0010	0.599	0.401	0.489	0.511	0.431	0.569	0.778	0.222	0.099	0.901
0011	0.180	0.820	0.841	0.159	0.878	0.122	0.099	0.901	0.216	0.784
0100	0.587	0.413	0.710	0.290	0.419	0.581	0.915	0.085	0.274	0.726
0101	0.298	0.702	0.455	0.545	0.443	0.557	0.439	0.561	0.397	0.603
0110	0.450	0.550	0.421	0.579	0.645	0.355	0.593	0.407	0.571	0.429
0111	0.827	0.173	0.680	0.320	0.763	0.237	0.446	0.554	0.356	0.644
1000	0.982	0.018	0.394	0.606	0.597	0.403	0.340	0.660	0.935	0.065
1001	0.775	0.225	0.443	0.557	0.789	0.211	0.415	0.585	0.076	0.924
1010	0.426	0.574	0.251	0.749	0.546	0.454	0.444	0.556	0.338	0.662
1011	0.804	0.196	0.597	0.403	0.566	0.434	0.814	0.186	0.242	0.758
1100	0.770	0.230	0.503	0.497	0.115	0.885	0.412	0.588	0.610	0.390
1101	0.511	0.489	0.940	0.060	0.600	0.400	0.918	0.082	0.566	0.434
1110	0.771	0.229	0.205	0.795	0.732	0.268	0.680	0.320	0.778	0.222
1111	0.939	0.061	0.310	0.690	0.735	0.265	0.470	0.530	0.598	0.402

Table 9: 5th-order Markov chains: Transition probabilities for all seeds.

Context	Chain 5.1		Chain 5.2		Chain 5.3		Chain 5.4		Chain 5.5	
	0	1	0	1	0	1	0	1	0	1
00000	0.479	0.521	0.764	0.236	0.448	0.552	0.887	0.113	0.439	0.561
00001	0.454	0.546	0.587	0.413	0.597	0.403	0.414	0.586	0.522	0.478
00010	0.283	0.717	0.443	0.557	0.027	0.973	0.192	0.808	0.513	0.487
00011	0.668	0.332	0.106	0.894	0.498	0.502	0.552	0.448	0.812	0.188
00100	0.359	0.641	0.580	0.420	0.365	0.635	0.880	0.120	0.452	0.548
00101	0.082	0.918	0.065	0.935	0.358	0.642	0.443	0.557	0.267	0.733
00110	0.359	0.641	0.505	0.495	0.470	0.530	0.335	0.665	0.723	0.277
00111	0.984	0.016	0.382	0.618	0.768	0.232	0.180	0.820	0.942	0.058
01000	0.649	0.351	0.675	0.325	0.769	0.231	0.307	0.693	0.231	0.769
01001	0.376	0.624	0.431	0.569	0.661	0.339	0.080	0.920	0.410	0.590
01010	0.475	0.525	0.764	0.236	0.476	0.524	0.502	0.498	0.496	0.504
01011	0.891	0.109	0.587	0.413	0.503	0.497	0.936	0.064	0.845	0.155
01100	0.996	0.004	0.598	0.402	0.894	0.106	0.555	0.445	0.843	0.157
01101	0.991	0.009	0.796	0.204	0.627	0.373	0.630	0.370	0.578	0.422
01110	0.953	0.047	0.549	0.451	0.564	0.436	0.455	0.545	0.062	0.938
01111	0.551	0.449	0.920	0.080	0.797	0.203	0.282	0.718	0.345	0.655
10000	0.175	0.825	0.536	0.464	0.594	0.406	0.418	0.582	0.390	0.610
10001	0.539	0.461	0.342	0.658	0.415	0.585	0.326	0.674	0.273	0.727
10010	0.819	0.181	0.215	0.785	0.384	0.616	0.743	0.257	0.760	0.240
10011	0.197	0.803	0.673	0.327	0.455	0.545	0.418	0.582	0.495	0.505
10100	0.624	0.376	0.892	0.108	0.819	0.181	0.734	0.266	0.791	0.209
10101	0.471	0.529	0.465	0.535	0.712	0.288	0.688	0.312	0.428	0.572
10110	0.712	0.288	0.544	0.456	0.730	0.270	0.897	0.103	0.489	0.511
10111	0.673	0.327	0.185	0.815	0.603	0.397	0.586	0.414	0.762	0.238
11000	0.635	0.365	0.531	0.469	0.435	0.565	0.786	0.214	0.983	0.017
11001	0.259	0.741	0.454	0.546	0.602	0.398	0.751	0.249	0.263	0.737
11010	0.400	0.600	0.828	0.172	0.138	0.862	0.520	0.480	0.812	0.188
11011	0.049	0.951	0.410	0.590	0.510	0.490	0.899	0.101	0.028	0.972
11100	0.635	0.365	0.356	0.644	0.917	0.083	0.995	0.005	0.512	0.488
11101	0.737	0.263	0.575	0.425	0.881	0.119	0.667	0.333	0.562	0.438
11110	0.787	0.213	0.649	0.351	0.495	0.505	0.946	0.054	0.654	0.346
11111	0.470	0.530	0.395	0.605	0.799	0.201	0.276	0.724	0.435	0.565

Table 10: 6th-order Markov chains: Transition probabilities for all seeds.

Context	Chain 6.1		Chain 6.2		Chain 6.3		Chain 6.4		Chain 6.5	
	0	1	0	1	0	1	0	1	0	1
000000	0.095	0.905	0.168	0.832	0.692	0.308	0.237	0.763	0.059	0.941
000001	0.716	0.284	0.577	0.423	0.361	0.639	0.907	0.093	0.277	0.723
000010	0.247	0.753	0.160	0.840	0.048	0.952	0.482	0.518	0.483	0.517
000011	0.275	0.725	0.860	0.140	0.057	0.943	0.491	0.509	0.788	0.212
000100	0.500	0.500	0.501	0.499	0.702	0.298	0.347	0.653	0.446	0.554
000101	0.210	0.790	0.516	0.484	0.854	0.146	0.835	0.165	0.465	0.535
000110	0.153	0.847	0.164	0.836	0.148	0.852	0.472	0.528	0.471	0.529
000111	0.251	0.749	0.629	0.371	0.570	0.430	0.491	0.509	0.661	0.339
001000	0.858	0.142	0.457	0.543	0.402	0.598	0.291	0.709	0.515	0.485
001001	0.711	0.289	0.581	0.419	0.718	0.282	0.663	0.337	0.564	0.436
001010	0.705	0.295	0.501	0.499	0.596	0.404	0.145	0.855	0.351	0.649
001011	0.345	0.655	0.487	0.513	0.223	0.777	0.665	0.335	0.508	0.492
001100	0.554	0.446	0.412	0.588	0.266	0.734	0.603	0.397	0.002	0.998
001101	0.334	0.666	0.242	0.758	0.375	0.625	0.283	0.717	0.462	0.538
001110	0.063	0.937	0.553	0.447	0.279	0.721	0.558	0.442	0.452	0.548
001111	0.254	0.746	0.473	0.527	0.333	0.667	0.349	0.651	0.136	0.864
010000	0.398	0.602	0.126	0.874	0.486	0.514	0.192	0.808	0.830	0.170
010001	0.444	0.556	0.851	0.149	0.440	0.560	0.460	0.540	0.214	0.786
010010	0.098	0.902	0.187	0.813	0.997	0.003	0.630	0.370	0.795	0.205
010011	0.253	0.747	0.222	0.778	0.177	0.823	0.318	0.682	0.260	0.740
010100	0.517	0.483	0.416	0.584	0.672	0.328	0.626	0.374	0.863	0.137
010101	0.908	0.092	0.704	0.296	0.307	0.693	0.413	0.587	0.611	0.389
010110	0.638	0.362	0.355	0.645	0.644	0.356	0.254	0.746	0.621	0.379
010111	0.654	0.346	0.456	0.544	0.769	0.231	0.464	0.536	0.254	0.746
011000	0.466	0.534	0.442	0.558	0.478	0.522	0.908	0.092	0.526	0.474
011001	0.637	0.363	0.442	0.558	0.637	0.363	0.482	0.518	0.551	0.449
011010	0.209	0.791	0.446	0.554	0.209	0.791	0.240	0.760	0.212	0.788
011011	0.218	0.782	0.875	0.125	0.218	0.782	0.648	0.352	0.024	0.976
011100	0.813	0.187	0.635	0.365	0.813	0.187	0.454	0.546	0.313	0.687
011101	0.734	0.266	0.521	0.479	0.734	0.266	0.107	0.893	0.665	0.335
011110	0.432	0.568	0.476	0.524	0.432	0.568	0.377	0.623	0.140	0.860
011111	0.844	0.156	0.774	0.226	0.844	0.156	0.417	0.583	0.613	0.387
100000	0.596	0.404	0.856	0.144	0.596	0.404	0.605	0.395	0.719	0.281
100001	0.457	0.543	0.468	0.532	0.457	0.543	0.800	0.200	0.122	0.878
100010	0.531	0.469	0.151	0.849	0.531	0.469	0.222	0.778	0.071	0.929
100011	0.567	0.433	0.575	0.425	0.567	0.433	0.454	0.546	0.302	0.698
100100	0.061	0.939	0.819	0.181	0.061	0.939	0.579	0.421	0.299	0.701
100101	0.496	0.504	0.764	0.236	0.496	0.504	0.434	0.566	0.357	0.643
100110	0.970	0.030	0.873	0.127	0.970	0.030	0.086	0.914	0.445	0.555
100111	0.637	0.363	0.633	0.367	0.637	0.363	0.684	0.316	0.086	0.914
101000	0.478	0.522	0.359	0.641	0.478	0.522	0.593	0.407	0.684	0.316
101001	0.874	0.126	0.840	0.160	0.874	0.126	0.274	0.726	0.633	0.367
101010	0.780	0.220	0.274	0.726	0.780	0.220	0.873	0.127	0.593	0.407
101011	0.699	0.301	0.169	0.831	0.699	0.301	0.840	0.160	0.503	0.497
101100	0.448	0.552	0.766	0.234	0.448	0.552	0.274	0.726	0.531	0.469
101101	0.766	0.234	0.112	0.888	0.766	0.234	0.699	0.301	0.667	0.333
101110	0.780	0.220	0.590	0.410	0.780	0.220	0.448	0.552	0.662	0.338
101111	0.112	0.888	0.746	0.254	0.112	0.888	0.766	0.234	0.334	0.666
110000	0.590	0.410	0.628	0.372	0.590	0.410	0.053	0.947	0.628	0.372
110001	0.746	0.254	0.307	0.693	0.746	0.254	0.504	0.496	0.279	0.721
110010	0.628	0.372	0.355	0.645	0.628	0.372	0.411	0.589	0.628	0.372
110011	0.307	0.693	0.977	0.023	0.307	0.693	0.486	0.514	0.301	0.699
110100	0.355	0.645	0.837	0.163	0.355	0.645	0.843	0.157	0.837	0.163
110101	0.977	0.023	0.776	0.224	0.977	0.023	0.048	0.952	0.776	0.224
110110	0.837	0.163	0.512	0.488	0.837	0.163	0.504	0.496	0.507	0.493
110111	0.776	0.224	0.301	0.699	0.776	0.224	0.411	0.589	0.301	0.699
111000	0.512	0.488	0.576	0.424	0.512	0.488	0.723	0.277	0.762	0.238
111001	0.301	0.699	0.530	0.470	0.301	0.699	0.918	0.082	0.339	0.661
111010	0.576	0.424	0.061	0.939	0.576	0.424	0.718	0.282	0.762	0.238
111011	0.530	0.470	0.673	0.327	0.530	0.470	0.718	0.282	0.339	0.661
111100	0.061	0.939	0.576	0.424	0.061	0.939	0.723	0.277	0.762	0.238
111101	0.673	0.327	0.530	0.470	0.673	0.327	0.918	0.082	0.339	0.661
111110	0.576	0.424	0.061	0.939	0.576	0.424	0.718	0.282	0.762	0.238
111111	0.530	0.470	0.673	0.327	0.530	0.470	0.718	0.282	0.339	0.661

Table 11: Training convergence analysis of the Transformers by Markov order.

Order	Convergence Point (iterations)		Final Loss	
	Mean	Std	Mean	Std
1	80	± 40	0.603	± 0.069
2	1,320	$\pm 1,067$	0.428	± 0.213
3	2,760	± 826	0.520	± 0.098
4	3,320	± 833	0.550	± 0.073
5	7,320	$\pm 1,816$	0.572	± 0.045
6	7,120	$\pm 3,643$	0.629	± 0.041

Table 12: Comparison of achieved vs. optimal loss by Markov order.

Order	Achieved Loss	Optimal Loss	Difference Factor*
	Mean $\pm$ Std		
1	0.603 $\pm$ 0.069	0.603	1.000x
2	0.428 $\pm$ 0.213	0.422	1.015x
3	0.520 $\pm$ 0.098	0.516	1.008x
4	0.550 $\pm$ 0.073	0.543	1.014x
5	0.572 $\pm$ 0.045	0.548	1.044x
6	0.629 $\pm$ 0.041	0.576	1.091x

*Difference Factor = Achieved Loss / Optimal Loss

B Figures

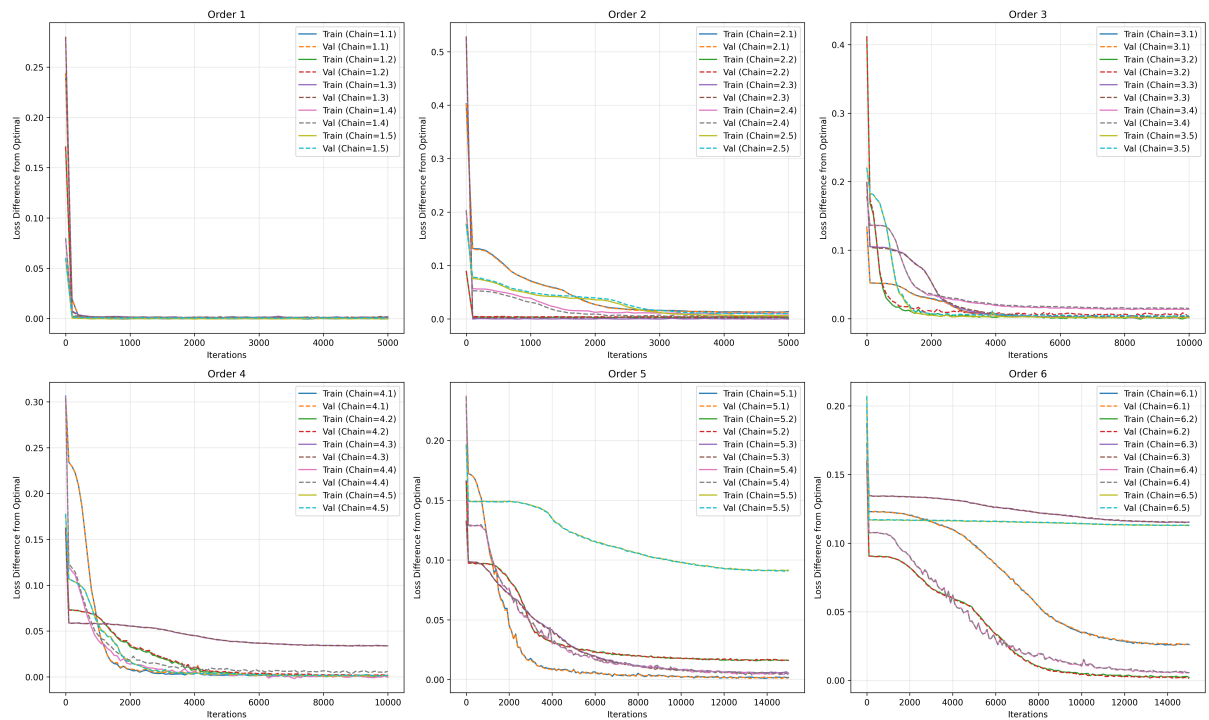


Figure 17: The loss curves for each Transformer model in relation to the optimum value. The associated transition matrices for each Markov chain can be seen in Tables 5–10.

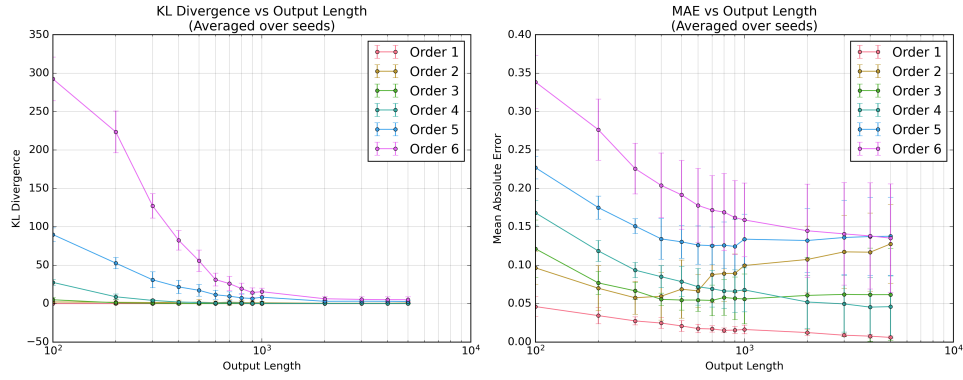


Figure 18: A Heatmap showing sequence length vs. metric score across all orders for both metrics.

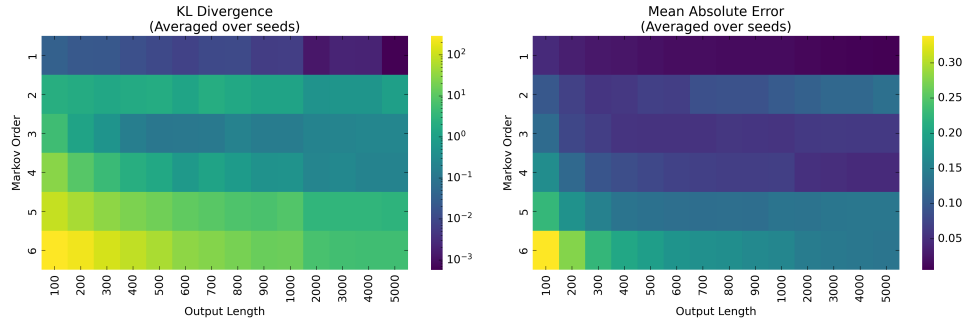


Figure 19: A heatmap showing Markov order vs. sequence length vs. metric score for both metrics.

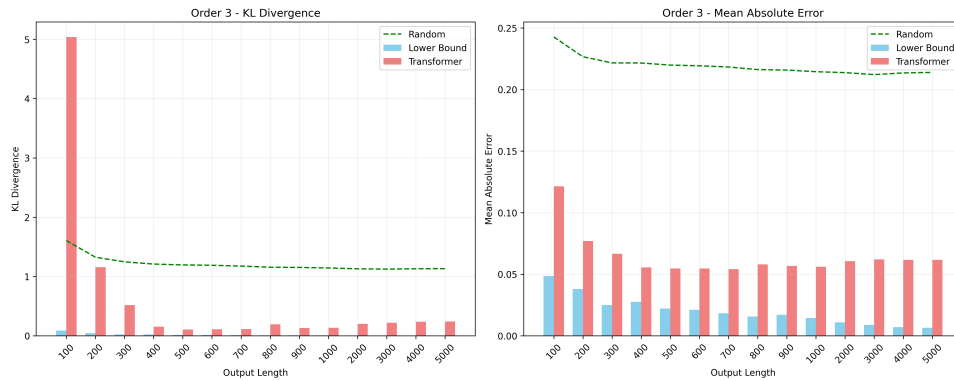


Figure 20: Markov order 3: Sequence length vs. the averaged KLD and MAE for the Transformers, the lower bound, and the random walk.

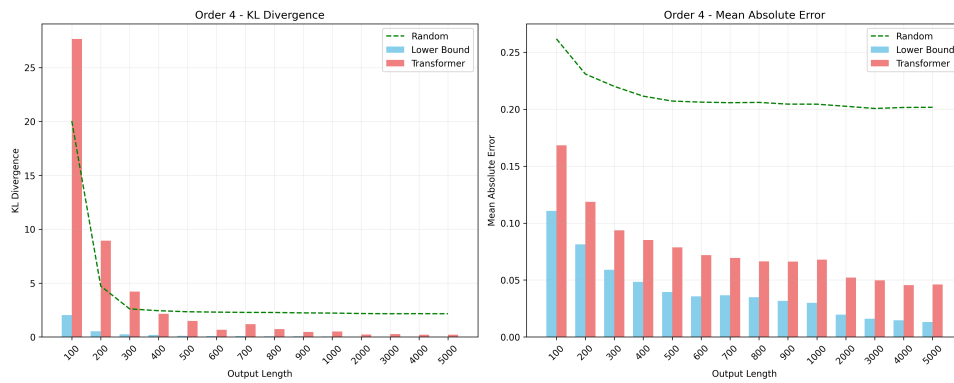


Figure 21: Markov order 4: Sequence length vs. the averaged KLD and MAE for the Transformers, the lower bound, and the random walk.

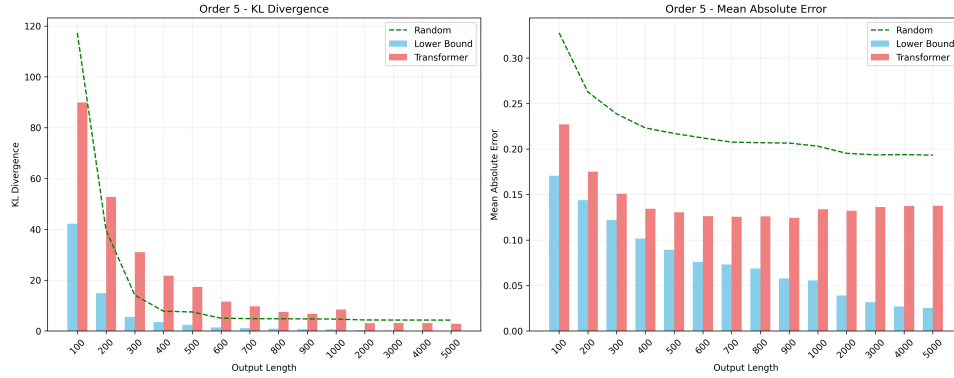


Figure 22: Markov order 5: Sequence length vs. the averaged KLD and MAE for the Transformers, the lower bound, and the random walk.

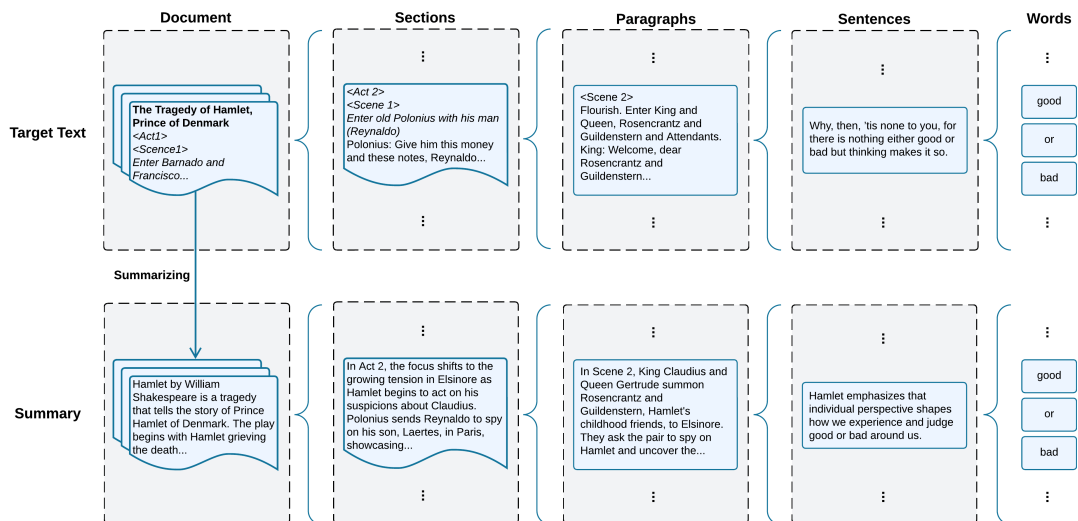


Figure 23: An illustration of a documents hierarchical structure that is transferred to the summary.

C Code Listings

```

1  import random
2  from collections import defaultdict, Counter
3
4  def generate_ngrams(text: str, n: int):
5      """
6      Generate n-grams from an input text.
7      """
8      words = text.split()
9
10     if n == 1:
11         # For unigrams, just count word frequencies
12         unigram_counts = Counter(words)
13         return dict(unigram_counts)
14
15     ngrams = defaultdict(list)
16     for i in range(len(words) - n + 1):
17         key = tuple(words[i:i + n - 1]) # n-1 words as the key
18         next_word = words[i + n - 1]   # nth word as the value
19         ngrams[key].append(next_word)
20
21     return ngrams
22
23 def generate_text(ngrams, n, m):
24     """
25     Generate text of length m using the n-gram model.
26     """
27     if n == 1: # unigrams -> frequency
28         words = list(ngrams.keys())
29         frequencies = list(ngrams.values())
30         total_count = sum(frequencies)
31         probabilities = [freq / total_count for freq in frequencies]
32         generated_words = random.choices(words, probabilities, k=m)
33         return ' '.join(generated_words)
34
35     # For n > 1, use n-grams
36     start = random.choice(list(ngrams.keys()))
37     generated_words = list(start)
38
39     for _ in range(m - len(start)):
40         current_key = tuple(generated_words[-(n - 1):]) # Get the last n-1 words as
41                                                         # the key
42         if current_key in ngrams:
43             next_word = random.choice(ngrams[current_key]) # Randomly choose the
44                                                         # next word
45             generated_words.append(next_word)
46         else:
47             break # Stop if no next word is found
48
49     return ' '.join(generated_words)

```

Listing 1: A Python code to estimate N-Grams and produce text.

```

1  def kl_divergence(chain1, chain2, epsilon=1e-10):
2      """
3      Calculate the KL-Divergence from two markov chains, ignoring undefined states in
4      chain2

```

```

5      Args:
6          chain1: torch.tensor - First Markov chain transition probabilities
7          chain2: torch.tensor - Second Markov chain transition probabilities
8          epsilon: float - Small value to avoid log(0)
9
10     Returns:
11         float - KL divergence from chain2 to chain1
12     """
13     # identify defined states in chain2 (sum of probabilities > 0)
14     defined_states = torch.sum(chain2, dim=1) > 0
15
16     # extract only the defined states and their corresponding states in chain1
17     p = chain1[defined_states]
18     q = chain2[defined_states]
19
20     # Add epsilon to p and q to avoid log(0)
21     p = p + epsilon
22     q = q + epsilon
23
24     # normalize both distributions
25     p = p / p.sum(dim=1, keepdim=True)
26     q = q / q.sum(dim=1, keepdim=True)
27
28     # calculate the KL-Divergence only for defined states
29     kl_div = torch.sum(p * torch.log(p / q))
30
31     return kl_div.item()

```

Listing 2: A Python code to calculate the Kullback-Leibler Divergence between two probability distributions.

```

1  def mean_error(chain1, chain2):
2      """
3          Calculate the mean absolute error of two Markov chains, ignoring undefined
4          states in chain2
5
6      Args:
7          chain1: torch.tensor - First Markov chain transition probabilities
8          chain2: torch.tensor - Second Markov chain transition probabilities
9
10     Returns:
11         float - Maximum absolute difference between corresponding probabilities
12     """
13     # identify defined states in chain2 (sum of probabilities > 0)
14     defined_states = torch.sum(chain2, dim=1) > 0
15
16     # extract only the defined states and their corresponding states in chain1
17     p = chain1[defined_states]
18     q = chain2[defined_states]
19
20     # Calculate the absolute differences for defined states
21     abs_diff = torch.abs(p - q)
22
23     # Calculate the mean error across all defined states
24     mean_error = torch.mean(abs_diff).item()
25
26     return mean_error

```

Listing 3: A Python code to calculate the Mean Absolute Error between two probability distributions.


```

1 def generate_transition_matrix(order: int, seed: int):
2     """
3     Generate random transition matrices for Markov chains of orders 1-6,
4     with 10 different matrices per order.
5
6     Args:
7         seed (int): Random seed for reproducibility
8
9     Returns:
10        dict: Dictionary with (order, id) tuples as keys and transition matrices as
11              values
12    """
13    if seed is not None:
14        torch.manual_seed(seed)
15
16    # Calculate number of states (2^order)
17    n_states = 2 ** order
18
19    # Generate random probabilities
20    matrix = torch.rand((n_states, 2))
21
22    # Normalize rows to sum to 1
23    row_sums = matrix.sum(dim=1, keepdim=True)
24    matrix = matrix / row_sums
25
26    return matrix

```

Listing 4: A Python code to generate Markov transition matrices.

```

1 import torch
2
3 def generate_sequence(transition_matrix: torch.tensor, order: int, length: int, seed
4 : int = None) -> list:
5     """
6     generates a binary sequence (array) from a given Markov transition matrix
7     """
8     if seed is not None:
9         torch.manual_seed(seed)
10
11    sequence = torch.zeros(length, dtype=torch.long)
12
13    # Generate first 'order' elements randomly
14    for i in range(order):
15        sequence[i] = torch.randint(0, 2, (1,))
16
17    # Generate remaining sequence using transition probabilities
18    for i in range(order, length):
19        # Calculate current state index from previous 'order' elements
20        state_idx = 0
21        for j in range(order):
22            state_idx = state_idx * 2 + sequence[i-order+j]
23
24        # Generate next bit based on transition probabilities
25        probs = transition_matrix[int(state_idx)]
26        sequence[i] = torch.bernoulli(probs[1])
27
28    return sequence.tolist()

```

Listing 5: A Python code to generate sequences drawn from a kth-order Markov chain.

```

1 import torch
2
3 def approximate_matrix(sequence: list,
4                       order: int) -> torch.Tensor:
5     """
6     Approximate a Markov transition matrix from a binary sequence.
7     """
8     # Initialize counts tensor
9     num_states = 2 ** order
10    counts = torch.zeros((num_states, 2))
11
12    # Count transitions
13    for i in range(len(sequence) - order):
14        # Get current state (last k bits)
15        state = sequence[i:i+order]
16        state_idx = int(''.join(map(str, state)), 2)
17
18        # Get next bit
19        next_bit = sequence[i+order]
20
21        # Update counts
22        counts[state_idx, next_bit] += 1
23
24    # Convert counts to probabilities
25    row_sums = counts.sum(dim=1, keepdim=True)
26    # Handle zero counts by setting uniform probabilities
27    row_sums[row_sums == 0] = 2
28    transition_matrix = counts / row_sums
29
30    return transition_matrix

```

Listing 6: A Python code to estimate a kth-order Markov chain for a given sequence.

```

1 import numpy as np
2
3 def calculate_optimal_loss(transition_matrix, order):
4     """
5     Calculate the optimal (theoretical minimum) cross-entropy loss for a Markov
6     chain of order k
7     """
8     # convert to a numpy array if not already
9     P_next = np.array(transition_matrix)
10    n_states = 2**order
11
12    # create the state transition matrix
13    P = np.zeros((n_states, n_states))
14
15    # fill the state transition matrix
16    for state in range(n_states):
17        for next_bit in [0, 1]:
18            # calculate the next state by shifting left and adding new bit
19            next_state = (state << 1) & (n_states - 1) | next_bit
20            # add the transition probability
21            P[state, next_state] = P_next[state, next_bit]
22
23    # find the stationary distribution using power iteration
24    pi = np.ones(n_states) / n_states # starting with uniform distribution
25    for _ in range(100):

```

```
25     pi_next = pi @ P
26     if np.allclose(pi, pi_next, rtol=1e-8):
27         break
28     pi = pi_next
29
30     # calculate the entropy rate
31     entropy_rate = 0.0
32     eps = 1e-10 # small constant to avoid log(0)
33
34     for state in range(n_states):
35         for next_bit in [0, 1]:
36             p = P_next[state, next_bit]
37             if p > eps:
38                 entropy_rate -= pi[state] * p * np.log(p)
39
40     return float(entropy_rate)
```

Listing 7: A Python code to calculate the optimal corss-entropy loss for a given kth-order Markov chain.